Using oTree: A Practical Guide

Nidhi Gyan Pandey

2026-04-06

Table of contents

Welcome **8**
Prerequisites . . . 8
A Note on This Book . . . 8

Let's Get Started! **9**

Setting Up Visual Studio Code **11**
macOS . . . 11
Installation . . . 11
Windows . . . 11
Installation . . . 11
Installing Extensions . . . 12
How to Install an Extension . . . 12
Required Extensions . . . 12
Recommended Extensions . . . 12

Installing Python and oTree **14**
Step 1: Check if Python is Already Installed . . . 14
Step 2: Install Python (Only if Needed) . . . 15
macOS . . . 15
Windows . . . 15
Step 3: Install oTree . . . 15
Step 4: Verify oTree Installation . . . 16
Troubleshooting . . . 16

Session 1 **17**

Getting Started: VS Code, Projects, and Apps **19**
Opening VS Code and navigating to the right folder . . . 19
Opening the terminal in VS Code . . . 19
Navigating folders: `cd` . . . 20
Finding your working directory: `pwd` . . . 20
Navigating to a folder using the VS Code terminal . . . 21

Starting a new oTree project: `otree startproject` . . . 22
What just got created? . . . 23
Starting a new app: `otree startapp` . . . 24
Running the server: `otree devserver` . . . 24
Resetting the database: `otree resetdb` . . . 25
Zipping your project: `otree zip` . . . 25
Summary of terminal commands . . . 26

Inside an oTree Project: Folder Structure and `settings.py` **27**
The oTree project folder . . . 27
The `settings.py` file . . . 29
`SESSION_CONFIGS`: Your session list . . . 30
`SESSION_CONFIG_DEFAULTS`: Project-wide defaults . . . 31
`PARTICIPANT_FIELDS` and `SESSION_FIELDS` . . . 31
`DEBUG` . . . 32
What you should never touch . . . 33
Summary . . . 35

Your First App: Adding, Registering, and Understanding `__init__.py` **36**
Step 1: Download and add the `demographic_survey` app . . . 36
Step 2: Register the app in `SESSION_CONFIGS` . . . 37
Step 3: Demo the app . . . 38
Step 4: Open `__init__.py` and understand the structure . . . 38
The import statement . . . 41
The `doc` string . . . 41
Class `C`: Constants . . . 42
Data model classes: `Subsession`, `Group`, and `Player` . . . 43
The `Player` class and model fields . . . 43
Page classes . . . 46
The `page_sequence` list . . . 47
Summary: The anatomy of `__init__.py` . . . 48

Session 2 **49**
From the Prisoner's Dilemma to the Stag Hunt . . . 50

Inside the Prisoner's Dilemma: `__init__.py` **51**
The import and `doc` . . . 53
Class `C`: constants for a two-player game . . . 53
Payoff constants with `cu()` . . . 54
How the payoff labels map to the game . . . 55
The data model classes . . . 55
`Subsession` and `Group` . . . 56

The `Player` class . . . 56
The functions: payoff logic . . . 56
`other_player`: finding the partner . . . 57
`set_payoff`: the payoff matrix as a dictionary . . . 57
`set_payoffs`: applying to the whole group . . . 59
The pages . . . 60
`Introduction`: a simple instruction page . . . 60
`Decision`: collecting the player's choice . . . 60
`ResultsWaitPage`: synchronising both players . . . 61
`Results`: showing the outcome . . . 61
The page sequence . . . 63
Comparing the survey app to the Prisoner's Dilemma . . . 63
Summary . . . 64

Inside the Prisoner's Dilemma: HTML Templates **65**
The basic template structure . . . 65
`Introduction.html`: a minimal page . . . 66
`{ include_sibling }`: reusing content across pages . . . 66
`instructions.html`: a reusable card . . . 67
HTML basics: tags and attributes . . . 68
The Bootstrap card . . . 69
The payoff table . . . 69
Displaying constants in the table . . . 70
Table structure: `colspan` and `rowspan` . . . 70
`Decision.html`: buttons as form submission . . . 71
Buttons instead of radio buttons . . . 72
The `{ chat }` widget . . . 73
`Results.html`: conditional display . . . 73
`{ if }` / `{ else }` / `{ endif }`: conditional display . . . 74
Displaying variables from `vars_for_template` . . . 75
File structure recap . . . 75
Summary . . . 76

Hands-On: Getting started with Assignment 2 **77**
Step 0: register the new app in `settings.py` . . . 77
Step 1: change `NUM_ROUNDS` to make this a repeated game . . . 78
Step 2: show the introduction only in round 1 . . . 79
Step 3: keep groups fixed across rounds . . . 80
Step 4: add a `per_round_payoff` field . . . 80
Step 5: build the game history in `vars_for_template` . . . 81
Creating an empty list . . . 82
Looping through previous rounds . . . 82
Getting the partner for each round . . . 83

Building each row . . . 83
Step 6: display the history table in the template . . . 84
`{ if game_history }`: checking if the list is non-empty . . . 85
`{ for row in game_history }`: looping through rows . . . 85
`<thead>` and `<tbody>` . . . 86
Step 7: update the Results page . . . 86
Step 8: add roles to your game . . . 87
Defining role constants . . . 87
Displaying roles in templates . . . 88
Displaying the current round . . . 88
Step 9: add a timer with a default choice . . . 88
Setting the timer . . . 89
Handling the timeout . . . 89
Step 10: add images and styled text . . . 90
Adding images with `{ static }` . . . 90
Styling text with CSS . . . 90
Step 11: test your changes . . . 91
What you have learned . . . 92
What comes next . . . 92

Assignment 2 Cheat Sheet **94**
Constants (`class C`) . . . 94
Required constants . . . 94
Currency values . . . 94
Role constants . . . 94
Model fields . . . 95
Field types . . . 95
Common field arguments . . . 95
Where to define fields . . . 95
Built-in player attributes . . . 96
Accessing other players and rounds . . . 96
Within the same round . . . 96
Across rounds . . . 96
Display values . . . 96
Grouping . . . 97
In `creating_session` . . . 97
Other grouping methods . . . 97
Page classes . . . 97
Basic page . . . 97
Page methods . . . 97
Wait pages . . . 98
Custom wait page template . . . 99
Timers . . . 99

Cross-app data: `PARTICIPANT_FIELDS` and `SESSION_FIELDS` . . . 99
Why you need them . . . 99
Declaring fields in `settings.py` . . . 100
Setting values . . . 100
Reading values in another app . . . 100
Initializing in `creating_session` . . . 100
Skipping rounds and apps . . . 101
Skipping to the next app with `app_after_this_page` . . . 101
Template syntax . . . 101
Displaying data . . . 101
Conditionals . . . 101
Loops . . . 102
Forms . . . 102
Buttons and navigation . . . 102
Including other files . . . 102
Static files (images, CSS, JS) . . . 103
The `settings.py` session config . . . 103
Python patterns quick reference . . . 103
If / elif / else . . . 103
For loops . . . 104
Dictionaries . . . 104
Lists . . . 104
List comprehensions . . . 104
Random numbers . . . 105
String formatting . . . 105
The `+=` operator . . . 105
Debugging . . . 105
`print()` in the terminal . . . 105
Common errors and fixes . . . 105
Useful links . . . 106

Session 3 **107**

Extra Features, HTML Templates, and Cross-App Data **108**
HTML templates in oTree . . . 108
Basic template structure . . . 109
Displaying variables . . . 109
Displaying form fields . . . 109
Using `if` statements in templates . . . 110
Using `for` loops in templates . . . 110
Extra feature: role-specific styling . . . 111
Approach 1: inline styles . . . 111
Approach 2: CSS classes (cleaner) . . . 111

Extra feature: custom wait page . . . 112
Extra feature: images on results pages . . . 112
Extra feature: Stag-Stag percentage on the summary screen . . . 113
Cross-app data flow . . . 113
What persists across apps automatically . . . 113
Data flow in this experiment . . . 113
The questionnaire app . . . 114
The session summary app . . . 114
Building the payoff matrix in HTML . . . 115
Summary . . . 116

Session 4 **117**

Using Claude for Coding and Deploying Your Experiment **118**
Using Claude to help with oTree coding . . . 118
What AI can do for you . . . 118
How to write effective prompts . . . 118
What AI is not good at . . . 119
Debugging with `print()` . . . 120
Common Python patterns for oTree . . . 120
If/elif/else . . . 121
For loops . . . 121
Lists . . . 121
Dictionaries . . . 122
String formatting . . . 122
Deploying your experiment . . . 122
Step 1: Zip your project . . . 122
Step 2: Upload to oTree Hub . . . 123
Step 3: Connect to Heroku . . . 123
Step 4: Set up rooms for a live session . . . 123
Step 5: Run the session . . . 124
Final checklist before submission . . . 124
Summary . . . 125

Useful Resources **126**
oTree . . . 126
Python . . . 127

References **128**

Welcome

This book serves as an A to Z resource for beginners learning to use oTree to conduct online behavioral experiments and surveys involving human subjects. No prior programming experience is assumed. It is designed to accompany ECON 227B Experimental Economics II lectures and course materials. The detailed syllabus and other course materials are available on the Canvas page for this course.

In this book, you will be introduced to coding simple experiments in the form of "applications" using oTree and deploying them on the Heroku server using oTree Hub. We will begin by installing oTree and Visual Studio Code on your personal laptops and familiarizing ourselves with the VS Code environment. Topics covered include creating an oTree application, executing common oTree tasks, testing code on a local server, running a session with human subjects, coding best practices, and using AI effectively to build your code. This book will also provide a curated list of resources by oTree creators and other users that you can draw on for your individual projects. The point is not to reinvent the wheel, but to stand on the shoulders of giants!

Prerequisites

Familiarity with Python is helpful but not required — we will build everything from scratch together. If you have never written a line of code before, do not worry. By the end of this course you will be comfortable navigating a coding environment and building your own experiments.

A Note on This Book

This book will grow alongside the course. Links to reference code and additional materials will be added as we progress through each session. If you are browsing ahead and find session pages that appear empty or incomplete, that is expected — content will be added as we reach those sessions in class.

Let's Get Started!

This section walks you through setting up your development environment before using oTree. Before we meet on April 1st for our first lab session, we request you to download and install the relevant software requirements.

Please ensure you have the following installed on your personal device:

1. Python — A general-purpose programming language.
2. oTree — An open-source Python-based framework used to build online interactive experiments.
3. Visual Studio Code — The Integrated Development Environment (IDE) we will be using for oTree.

Step-by-step instructions for each are provided in the sections below.

Setting Up Visual Studio Code

Visual Studio Code (VS Code) is a free, lightweight code editor we will use throughout this course to write and run oTree code. Follow the instructions below for your operating system.

macOS

Installation

1. Go to https://code.visualstudio.com/.
2. Click **Download for Mac**.
3. Once downloaded, open the `.zip` file — this extracts **Visual Studio Code.app**.
4. Drag **Visual Studio Code.app** into your **Applications** folder.
5. Open VS Code from your Applications folder or Spotlight Search (`Cmd + Space`, type "VS Code").

> **Tip**
>
> To keep VS Code easily accessible, right-click its icon in the Dock and select **Options → Keep in Dock**.

Windows

Installation

1. Go to https://code.visualstudio.com/.
2. Click **Download for Windows**.
3. Run the downloaded `.exe` installer.
4. Follow the setup wizard and click **Install**, then **Finish**.
5. Launch VS Code from the Start Menu or Desktop shortcut.

Installing Extensions

Extensions add functionality to VS Code. For this course, you will need the following:

How to Install an Extension

1. Open VS Code.
2. Click the **Extensions** icon in the left sidebar (or press `Ctrl+Shift+X` on Windows / `Cmd+Shift+X` on Mac).
3. Type the extension name in the search bar.
4. Click **Install**.

Required Extensions

Extension	Publisher	Purpose
Python	Microsoft	Core Python language support
Pylance	Microsoft	Improved Python code suggestions and error detection
Python Debugger	Microsoft	Step-through debugging for your oTree code

Recommended Extensions

Extension	Publisher	Purpose
Django	Baptiste Darthenay	Syntax highlighting for oTree HTML templates, which are Django-based
Python Environments	Microsoft	Manage Python virtual environments for your oTree projects
vscode-icons	VSCode Icons Team	Adds file and folder icons for easier project navigation

> **Note**
>
> If you already have VS Code installed, open the Extensions panel and check which of these you already have. You only need to install the missing ones.

Installing Python and oTree

oTree is built on Python, so we need to install Python first before installing oTree. Follow the steps below for your operating system.

Step 1: Check if Python is Already Installed

Before downloading anything, check if Python is already on your system. Open your Terminal (macOS) or Command Prompt (Windows) and run:

macOS — run in Terminal:

```
python3 --version
```

Windows — run in Command Prompt:

```
python --version
```

If you see something like `Python 3.10.x` or higher, you already have Python and can **skip to Step 2**.

> **Note**
>
> If your terminal prompt starts with `(base)`, you have Anaconda or Miniconda installed. This is perfectly fine — Python is already included and you can skip directly to Step 2.

Step 2: Install Python (Only if Needed)

macOS

1. Go to https://www.python.org/downloads/.
2. Click **Download Python** — it will suggest the latest version.
3. Open the downloaded `.pkg` file and follow the installer.
4. Once installed, verify by running:

```
python3 --version
```

You should see something like `Python 3.12.x`. If you do, Python is successfully installed.

Windows

1. Go to https://www.python.org/downloads/.
2. Click **Download Python** — it will suggest the latest version.
3. Run the downloaded `.exe` installer.
4. **Important:** On the first screen, check the box that says **Add Python to PATH** before clicking Install Now.

> ⚠ Warning
>
> If you skip checking **Add Python to PATH**, Python will not be recognized in the terminal and the next steps will not work. If you already installed Python without this box checked, uninstall it and reinstall with the box checked.

5. Once installed, verify by running:

```
python --version
```

You should see something like `Python 3.12.x`. If you do, Python is successfully installed.

Step 3: Install oTree

Once Python is confirmed, install oTree using `pip`, Python's built-in package installer.

macOS — run in Terminal:

```
pip3 install otree --upgrade
```

Windows — run in Command Prompt:

```
pip install otree --upgrade
```

This may take a minute or two as pip downloads and installs oTree and its dependencies. The `--upgrade` flag ensures you get the latest version even if oTree was previously installed on your system.

Step 4: Verify oTree Installation

Once the installation is complete, verify it by running the following in your Terminal or Command Prompt:

```
oTree --version
```

You should see `6.0.13` or similar. If you do, you are all set!

Troubleshooting

Problem	Fix
`python3: command not found` (macOS)	Install Python from python.org
`python: command not found` (Windows)	Reinstall Python and check **Add Python to PATH**
`pip3: command not found` (macOS)	Run `python3 -m ensurepip` in Terminal
`oTree: command not found`	Run `pip3 install otree --upgrade` again and restart Terminal

> **Tip**
>
> If you are stuck before the first session, you can use ZotGPT — UCI's free AI platform — to troubleshoot. Select the latest Claude model available and describe your error message. It is particularly good at solving software installation issues.
> You can also bring your laptop to the first lab session on April 1st and we will troubleshoot together.

Session 1

In this session, we build the foundation for everything that follows in this course. We start by getting comfortable with VS Code and the terminal, then move into creating and understanding the structure of an oTree project, and finish with a hands-on exercise where you will download, configure, and run a real oTree app.

By the end of this session, you will be able to:

- Navigate your file system using the terminal and create a new oTree project and app from scratch.
- Read and make sense of what is inside an oTree project folder, and understand the key components of the `settings.py` file.
- Download an app shared by someone else, add it to your project, update `settings.py` to include it, and run a working demo in your browser.

This session has three parts:

Part	Title	What you will do
Getting Started	Getting Started: VS Code, Projects, and Apps	Learn the core terminal commands for working with oTree
Inside an oTree Project	Inside an oTree Project: Folder Structure and `settings.py`	Understand what oTree creates and how to configure it
Hands-On	Hands-On: Adding and Running Your First App	Download, configure, and demo a shared oTree app

Getting Started: VS Code, Projects, and Apps

This part covers the core terminal commands you will use every time you work in oTree. By the end of this part, you will be able to create a new oTree project and app from scratch, run the development server, and navigate your project folder in VS Code.

Opening VS Code and navigating to the right folder

Before creating an oTree project, decide where on your computer you want to store your work. A good habit is to keep all your oTree projects inside a dedicated folder — for example, a folder called `oTree_lab` inside your Documents folder.

To open that folder in VS Code:

1. Open VS Code
2. Click **File** → **Open Folder**
3. Navigate to your chosen folder and click **Open**

You should now see the folder listed in the Explorer panel on the left side of the screen.

Opening the terminal in VS Code

Almost everything in oTree is done through the terminal. In VS Code, open a terminal by clicking **Terminal** → **New Terminal** from the top menu bar. A terminal panel will appear at the bottom of the screen.

> **What is the terminal?**
>
> The terminal is a text-based interface where you type commands directly to your computer. You will use it constantly in oTree — to navigate between folders, create projects and apps, and run the server.

Navigating folders: `cd`

The terminal always has a "current directory" — the folder it is currently pointed at. Before running any oTree commands, make sure the terminal is pointed at the right place.

The command `cd` (short for "change directory") is how you move around.

Navigate into a folder:

```
cd Documents/oTree_lab
```

Go back to your home directory:

```
cd ~
```

> ⚠ Avoid spaces in folder names
>
> If any folder in your path has a space in its name, wrap the entire path in quotes:
>
> ```
> cd "Documents/my projects/oTree_lab"
> ```
>
> The safest habit is to never use spaces in folder or file names at all — use underscores instead (e.g., `oTree_lab`).

Finding your working directory: `pwd`

Before running any oTree command, you need to know exactly where the terminal is pointed. The command `pwd` (print working directory) tells you the full path of your current location.

```
pwd
```

You will see output like this:

```
/Users/nidhipandey/Documents/GitHub/all-otree-code
```

This is the single most useful command for diagnosing navigation errors. If a `cd` command fails with "no such file or directory," run `pwd` first to confirm where you actually are before trying again.

> ⚠ The Explorer panel and the terminal are not always in sync
>
> The VS Code Explorer panel shows the folder you have **open in VS Code**. The terminal shows the folder it is **currently pointed at**. These are two separate things and they can easily get out of sync — especially if you open the terminal from a different location or run `cd ~` and navigate elsewhere.
> Always check `pwd` if the terminal and the Explorer panel seem to be showing different things.

Navigating to a folder using the VS Code terminal

The safest and most reliable way to make sure the terminal is pointed at the right folder is to let VS Code do it for you.

The recommended workflow:

1. Open VS Code
2. Click **File** → **Open Folder** and select the folder you want to work in — for example, `all-otree-code`
3. Click **Terminal** → **New Terminal**

VS Code will automatically open the terminal pointed at the folder you have open in the Explorer panel. You will see the folder name in the terminal prompt confirming this.

From there, if you want to navigate into a subfolder — for example `oTree_lab` — just run:

```
cd oTree_lab
```

This works every time because the terminal is already inside the parent folder. You do not need to type out the full path.

> ❗ Always open a new terminal from inside VS Code
>
> If you open a terminal from outside VS Code — for example from macOS Spotlight or the Applications folder — it will start pointed at your home directory `~`, not at your project folder. You will then have to navigate manually using the full path, which is easy to get wrong.
> The habit to build: **open VS Code first, open the right folder, then open the terminal from inside VS Code.**

When `cd` says "no such file or directory"

This error does not always mean the folder does not exist. It usually means the terminal cannot see it from its current location. Run `pwd` to check where you are, then build the correct path from there. For example, if `pwd` returns `/Users/yourname/Documents/GitHub/all-otree-code`, then a folder called `oTree_lab` inside it is reachable with:

```
cd oTree_lab
```

But a folder somewhere else entirely on your computer will not be reachable with just its name. You must navigate to its parent folder first, or use the full path:

```
cd /Users/yourname/Documents/GitHub/all-otree-code/oTree_lab
```

Starting a new oTree project: `otree startproject`

Project vs. App — know the difference

An oTree **project** is the top-level container for your experiment. It holds the `settings.py` file and all of your apps. You will typically only create one project per course or research project. This is a rare task; most of the times you would be creating new apps within an existing project.
An oTree **app** is a single task or module within your experiment (e.g., a survey, a game, a payment screen). You will create new apps regularly.
Make sure you know which one you need before running a command.

Navigate to your desired folder in the terminal, then run:

```
otree startproject PROJECT_NAME
```

Replace `PROJECT_NAME` with a name of your choice — no spaces or special characters. For example:

```
otree startproject oTree_lab
```

oTree will ask whether you want sample games included:

```
Include sample games? (y or n):
```

Type y and press Enter. This is useful for your first project as it gives you working examples to explore.

Once the project is created, navigate into it:

```
cd oTree_lab
```

What just got created?

Click on the image below to see what the oTree project folder that you created looks like.

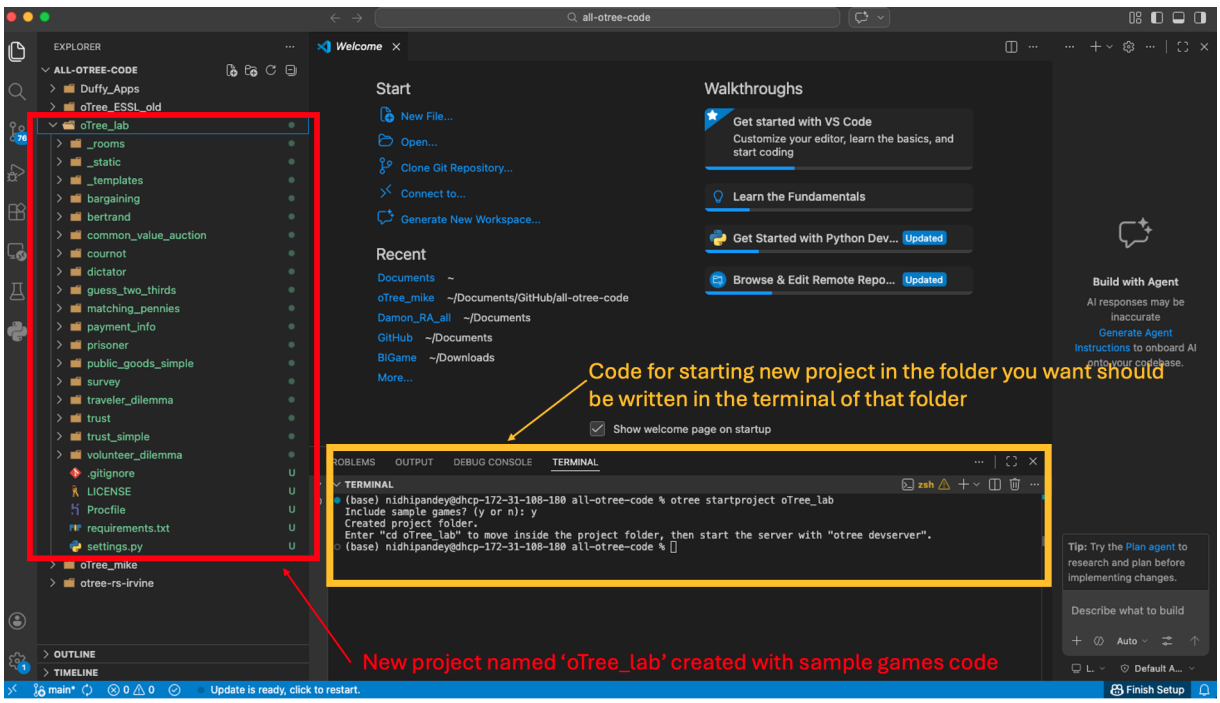

Figure 0.1: VS Code after running `otree startproject oTree_lab`. The Explorer panel (left) shows the newly created project folder containing the `settings.py` file and all sample app folders. The terminal at the bottom shows the full sequence of commands run to create and navigate into the project. Note: the code for starting a new project should be written in the terminal of the folder you want the project to live in.

Open the Explorer panel in VS Code and look at your new project folder. Here are the key things to know:

Item	What it is
`settings.py`	The control center for the entire project. You will edit this regularly.
`_rooms/`	Where room configuration files go. Do not touch unless instructed.
`_static/`	Where shared static files (images, CSS, JS) go.
`_templates/`	Where shared HTML templates go.
Sample app folders	The sample games oTree installed for you.

Starting a new app: `otree startapp`

Make sure your terminal is pointed at your project folder, then run:

```
otree startapp APP_NAME
```

For example:

```
otree startapp my_survey
```

This creates a new folder called `my_survey` inside your project. Inside it you will find:

- `__init__.py` — the Python file where all the logic for your app lives
- No HTML files yet — you will create those yourself as you build out your pages

> **Name your apps carefully**
>
> App names become part of the URL that participants see during your experiment. Use short, lowercase names with underscores and no special characters. Avoid names that reveal anything about your experimental design (e.g., do not call an app `treatment_group`).

Running the server: `otree devserver`

To preview your project in the browser, run:

```
otree devserver
```

You will see output like this in the terminal:

```
Open your browser to http://localhost:8000/
To quit the server, press Control+C.
```

Open your browser and go to `http://localhost:8000/demo` to see the oTree Demo page. All the sample games will be listed there. Click on any one to try it out.

> ⚠ Always quit the server when you are done
>
> When you are finished, go back to the terminal and press **Ctrl + C** to stop the server. Do not just close the browser window — always quit the server first.

Resetting the database: `otree resetdb`

Whenever you make changes to model variables in your app or update `settings.py`, you need to reset the database before running the server again:

```
otree resetdb
```

oTree will ask you to confirm:

```
This will delete and recreate your database.
Proceed? (y or n): y
```

> ❗ Do not reset if you have real session data
>
> Resetting the database permanently deletes all session data. If you have collected data from real participants, download it **before** running `otree resetdb`. You cannot recover it afterward.

Zipping your project: `otree zip`

The `otree zip` command packages your entire oTree project into a single `.otreezip` file. This is useful for sharing your project with collaborators, submitting it for a class, or uploading it to a server for deployment.

Make sure your terminal is pointed at your project folder, then run:

```
otree zip
```

oTree will create a file called `PROJECT_NAME.otreezip` in the same folder. For example, if your project is called `my_experiment`, the output will be:

```
Created my_experiment.otreezip
```

You can share this file directly with anyone who has oTree installed. To unzip and run it, they simply run:

```
otree unzip my_experiment.otreezip
```

> When to zip
>
> Zip your project whenever you want to share it or submit it. It is also a good habit to zip and save a copy of your project before making major changes — think of it as a quick manual backup.

Summary of terminal commands

Command	What it does
`pwd`	Print the full path of your current directory
`cd FOLDER_NAME`	Navigate into a folder
`cd ~`	Return to your home directory
`otree startproject NAME`	Create a new oTree project
`otree startapp NAME`	Create a new app inside a project
`otree devserver`	Start the local development server
`otree resetdb`	Reset the database
`otree zip`	Package the project into a `.otreezip` file
`otree unzip FILE.otreezip`	Unpack a zipped project
`Ctrl + C`	Stop the server

Inside an oTree Project: Folder Structure and `settings.py`

Now that you have created your first oTree project, it is time to understand what is actually inside it. Every decision about how your experiment runs — which apps are included, how many participants can demo it, what the payment structure looks like — flows through one file: `settings.py`. This part explains the project folder layout and walks through the key components of `settings.py` so you know what to edit, what to leave alone, and why.

The oTree project folder

When you run `otree startproject` and include sample games, oTree generates a ready-to-use project folder. Click on the image below to see what that looks like in VS Code.

Here is a breakdown of every item you will see in a new oTree project:

Item	What it is	Do you edit it?
`settings.py`	The control center for the entire project	Yes — regularly
`_rooms/`	Stores room configuration files for live sessions	Only when instructed
`_static/`	Shared static files (images, CSS, JS) used across apps	As needed
`_templates/`	Shared HTML templates used across apps	As needed
App folders (e.g., `survey/`)	Each folder is one oTree app	Yes — this is where you build
`requirements.txt`	Lists Python packages the project depends on	Rarely
`Procfile`	Server configuration file	Do not touch
`LICENSE`	License file	Do not touch

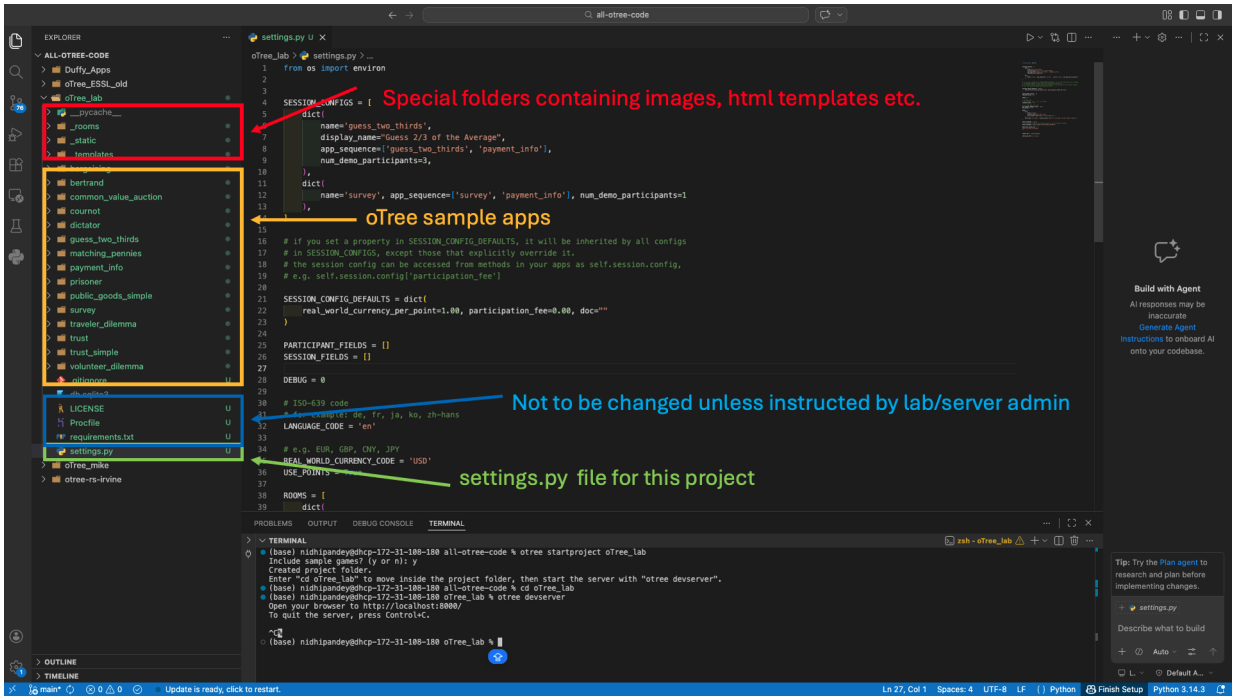

Figure 0.1: The oTree project folder `oTree_lab` open in VS Code. The Explorer panel shows the full contents of the project: the special system folders (`_rooms`, `_static`, `_templates`), the pre-installed sample app folders (such as `bargaining`, `prisoner`, `survey`, and others), and the `settings.py` file at the bottom of the list. The terminal confirms that the project was created and the working directory is now inside `oTree_lab`.

> ⚠ **Special folders vs. app folders**
>
> The three folders that begin with an underscore — `_rooms/`, `_static/`, and `_templates/` — are system folders, not apps. Do not confuse them with your app folders. Never delete or rename them unless instructed by your lab administrator.

The `settings.py` file

The `settings.py` file is the control center for your entire oTree project. Every app you want to run, every payment parameter, and every session configuration lives here. You will edit this file regularly throughout the course.

Click on the image below to see what a typical `settings.py` file looks like.

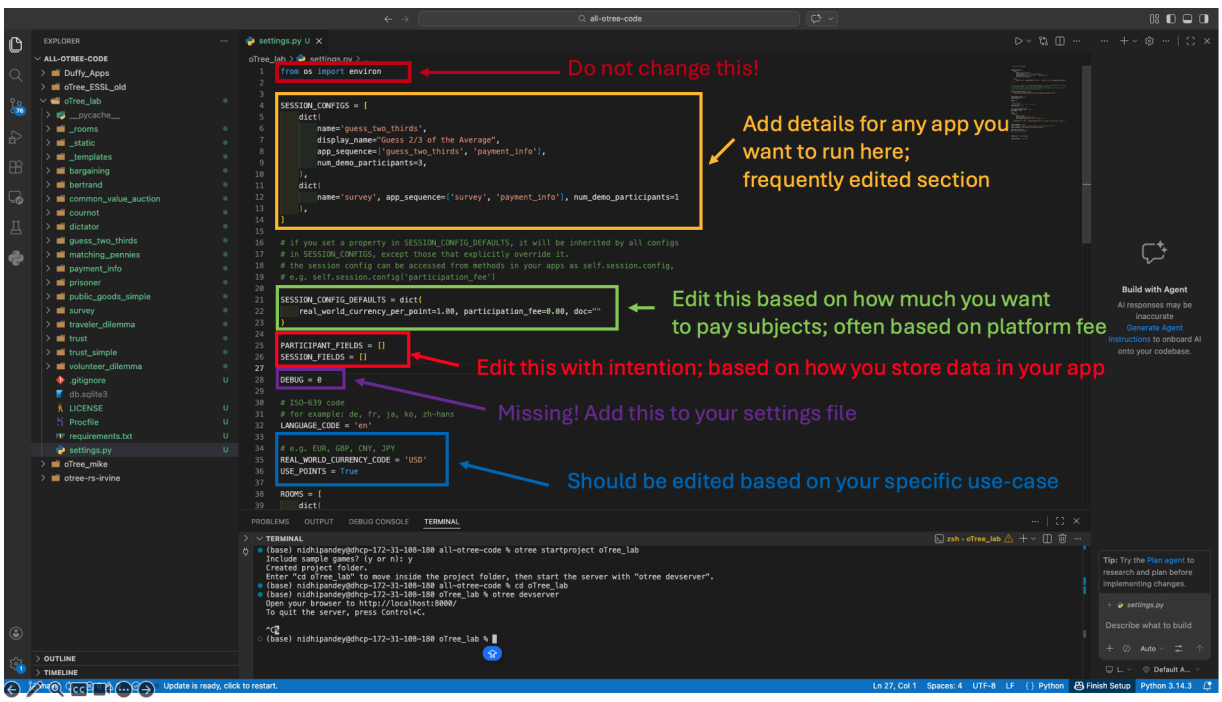

Figure 0.2: The `settings.py` file for an oTree project open in VS Code. The file contains several distinct sections: `SESSION_CONFIGS` (edited regularly), `SESSION_CONFIG_DEFAULTS` (edit with intention as needed), `PARTICIPANT_FIELDS` and `SESSION_FIELDS` (edited with intention as needed), and system-level settings such as `SECRET_KEY` and `INSTALLED_APPS` (never touch these). Each section serves a different purpose in controlling how your experiment runs.

`SESSION_CONFIGS`: Your session list

`SESSION_CONFIGS` is a Python list of dictionaries. Each dictionary defines one session — one experiment you can run or demo from the admin interface. Think of it as the menu of sessions available in your project.

Here is what a `SESSION_CONFIGS` list looks like:

```
SESSION_CONFIGS = [
    dict(
        name='guess_two_thirds',
        display_name="Guess 2/3 of the Average",
        app_sequence=['guess_two_thirds', 'payment_info'],
        num_demo_participants=3,
    ),
    dict(
        name='survey',
        app_sequence=['survey', 'payment_info'],
        num_demo_participants=1,
    ),
]
```

Every session dictionary requires three key-value pairs:

Key	What it does
`name`	A unique internal identifier for the session. No spaces or special characters.
`app_sequence`	A Python list of app names in the order participants will complete them.
`num_demo_participants`	How many participants are created when you run a demo of this session.

> **Choosing `num_demo_participants`**
>
> Set this number based on your experimental design. If an app groups participants in pairs, use a multiple of 2. If it groups them in threes, use a multiple of 3. For individual decision-making tasks, 1 is enough to test the app.

You can also include optional keys in any session dictionary:

Optional key	What it does
`display_name`	The name shown in the admin interface. If omitted, `name` is used.
`real_world_currency_per_point`	Overrides the default exchange rate for this session only.
`participation_fee`	Overrides the default participation fee for this session only.

`SESSION_CONFIG_DEFAULTS`: Project-wide defaults

`SESSION_CONFIG_DEFAULTS` is a single dictionary that sets default values inherited by all sessions unless a session dictionary explicitly overrides them. The two most commonly used defaults are `real_world_currency_per_point` and `participation_fee`.

```
SESSION_CONFIG_DEFAULTS = dict(
    real_world_currency_per_point=1.00,
    participation_fee=10.00,
    doc=""
)
```

If you want one specific session to use a different exchange rate or participation fee, simply add that key to that session's dictionary in `SESSION_CONFIGS` and it will override the default for that session only.

> **!** Lab participation fees
>
> If you run sessions through the ESSL lab, you are bound by the lab's participation fee policy. Make sure the `participation_fee` in your settings reflects the approved rate before running any real sessions.

`PARTICIPANT_FIELDS` and `SESSION_FIELDS`

These two settings are lists that become relevant in more advanced experiments where you need to carry data from one app to the next — for example, storing a participant's treatment assignment in the first app and reading it back in the payment screen.

- Use `PARTICIPANT_FIELDS` for data that is specific to an individual participant (e.g., their assigned treatment condition).
- Use `SESSION_FIELDS` for data that applies equally to everyone in the session.

For now, you can leave these as empty lists:

```
PARTICIPANT_FIELDS = []
SESSION_FIELDS = []
```

You will learn how to use them properly in later sessions when your experiments involve multiple apps.

`DEBUG`

You do not currently have this in your settings.py file. Add DEBUG = 0 to your file settings.py under `SESSION_FIELDS`:

```
PARTICIPANT_FIELDS = []
SESSION_FIELDS = []

DEBUG = 0
```

What `DEBUG` does

`DEBUG` controls whether oTree runs in development mode or production mode.

Value	Mode	What it means
`DEBUG = 1`	Development mode	Extra tools and error information are turned on
`DEBUG = 0`	Production mode	Clean participant-facing interface, no extra tools

Why `DEBUG = 1` is useful when building your app

When you are writing and testing your code, setting `DEBUG = 1` gives you two very useful advantages:

1. **Auto-advance buttons** — On every page, oTree adds a small button that lets you skip past that page without manually filling in any form fields — pages are submitted with empty or default values instead. This lets you click through the entire experiment in seconds to check that the page flow works correctly, without having to fill in every input every time.
2. **Detailed error messages** — If your Python code crashes, oTree shows you a full error traceback directly in the browser instead of just a blank page. This makes it much faster to find and fix problems in your code.

These two features make the development cycle dramatically faster. Instead of filling in every form field on every page just to check one thing at the end of the experiment, you can click through the whole app in a few seconds.

Why you must switch to `DEBUG = 0` before running real sessions

When `DEBUG = 1`, participants can also see the auto-advance buttons and skip past pages without submitting their answers. This means your data will be incomplete or entirely missing. Always confirm that `DEBUG = 0` before inviting any real participants.

> ⚠ Do not run real sessions with `DEBUG = 1`
>
> If `DEBUG = 1` is on during a live session, participants will see a skip button on every page that lets them bypass your forms — if they use it, their answers will not be recorded. Always confirm `DEBUG = 0` before starting any real data collection.

> 💡 A useful development habit
>
> While building and testing your app: set `DEBUG = 1`. Before running any real session: set `DEBUG = 0`. This single switch is one of the most common sources of data collection errors for new oTree users — do not forget it.

What you should never touch

Some parts of `settings.py` must not be modified under any circumstances:

Setting	Why you leave it alone
`from os import environ`	System import required by oTree — removing it breaks the project
`SECRET_KEY`	Security key for the project — changing it will break everything
`INSTALLED_APPS`	Registers oTree with Django — do not modify
`ADMIN_PASSWORD`	Managed via environment variables by your administrator

> ⚠ When in doubt, do not touch it
>
> If a line in `settings.py` is not covered in this guide, leave it exactly as it is. Only edit the parts you understand. When in doubt, ask before making any changes.

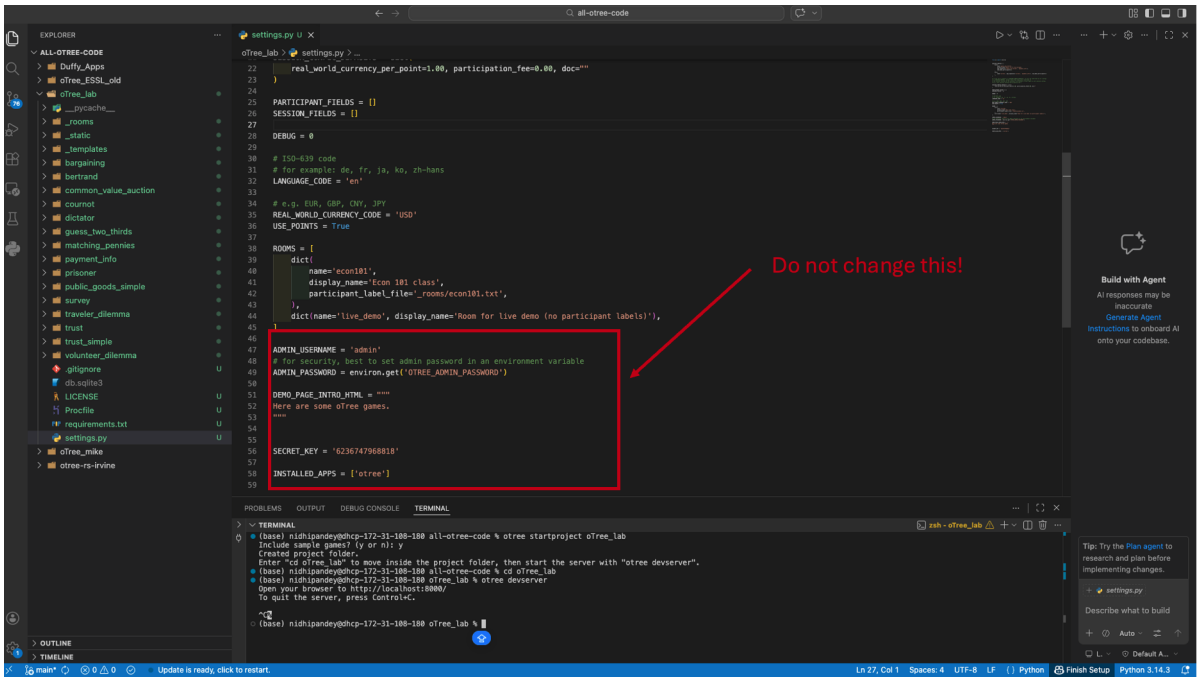

Figure 0.3: The end of the `settings.py` file for an oTree project open in VS Code. Do not change system-level settings such as `SECRET_KEY` and `INSTALLED_APPS`. Each section serves a different purpose in controlling how your experiment runs.

Summary

Section	What it controls	How often you edit it
`SESSION_CONFIGS`	Which sessions and apps are available	Regularly
`SESSION_CONFIG_DEFAULTS`	Default payment and session parameters	Occasionally
`PARTICIPANT_FIELDS`	Cross-app participant-level data	When needed
`SESSION_FIELDS`	Cross-app session-level data	When needed
`USE_POINTS`	If `True`, earnings are shown in points and converted using the exchange rate. If `False`, real currency is used directly	When needed — based on your design
`DEBUG`	`0` = production mode (normal). `1` = debug mode (shows extra info on participant pages)	Leave at `0` unless developing app or troubleshooting
System settings	Security, server configuration	Never

Your First App: Adding, Registering, and Understanding `__init__.py`

In this session you will add a pre-built app to your oTree project, register it in `settings.py`, demo it in the browser, and then open the code to understand how an oTree app is structured from the inside. By the end of this part you will be able to read and navigate any `__init__.py` file and understand what each component does.

Step 1: Download and add the `demographic_survey` app

The example code for this course is stored in a shared Google Drive folder. Navigate to the link below and download the file called `demographic_survey.zip`:

Example Code — Google Drive

Once downloaded:

1. Locate `demographic_survey.zip` in your Downloads folder
2. Double-click the file to unzip it — this will produce a folder called `demographic_survey`
3. Move the `demographic_survey` folder into your oTree project folder — for example, into `oTree_lab`

After moving it, open VS Code and check the Explorer panel. You should now see `demographic_survey` listed alongside the other app folders inside your project.

> ⚠ The folder must be inside the project folder
>
> The `demographic_survey` folder must sit directly inside your oTree project folder — at the same level as `settings.py` and the other app folders. If it is placed anywhere else, oTree will not be able to find it.

Step 2: Register the app in `SESSION_CONFIGS`

Adding the folder to your project is not enough on its own. oTree only runs apps that are listed in `SESSION_CONFIGS` inside `settings.py`. Open `settings.py` and add the following dictionary to your `SESSION_CONFIGS` list:

```
SESSION_CONFIGS = [
    # ... your existing session dictionaries ...

    dict(
        name='survey_leveled_up',
        display_name="End of experiment questionnaires",
        app_sequence=['demographic_survey'],
        num_demo_participants=1,
        doc="Standard end of experiment questionnaires."
    ),
]
```

Notice the five key-value pairs in this dictionary:

Key	Value	What it does
`name`	`'survey_leveled_up'`	Internal identifier used by the server
`display_name`	`"End of experiment questionnaires"`	The name shown in the admin interface
`app_sequence`	`['demographic_survey']`	The list of apps in this session — just one here
`num_demo_participants`	`1`	One participant is enough to demo a survey
`doc`	`"Standard end of experiment questionnaires."`	A short note describing the session — optional but good practice

Save `settings.py` after making the change.

> **❗ Reset the database after editing `settings.py`**
>
> Any time you edit `settings.py` — including adding a new session — you must reset the database before running the server. In the terminal, make sure you are inside your project folder, then run:
>
> ```
> otree resetdb
> ```
>
> Confirm with `y` when prompted.

Step 3: Demo the app

With the database reset, start the development server:

```
otree devserver
```

Open your browser and go to `http://localhost:8000/demo`. You should now see `End of experiment questionnaires` listed on the Demo page. Click on it to launch a demo session and work through the app as a participant would. When you are done, press **Ctrl + C** in the terminal to stop the server.

Step 4: Open `__init__.py` and understand the structure

Every oTree app has exactly one `__init__.py` file. This single Python file contains everything the app needs to run: the constants, the data models, the page logic, and the page sequence. Open `demographic_survey/__init__.py` in VS Code. Here is the full file:

```
from otree.api import *

doc = """
Standard, end-of-session survey.
Includes Cognitive Reflection Task and demographic survey.
"""

class C(BaseConstants):
    NAME_IN_URL = 'survey_experiment'
    PLAYERS_PER_GROUP = None
    NUM_ROUNDS = 1


class Subsession(BaseSubsession):
    pass


class Group(BaseGroup):
    pass


class Player(BasePlayer):
    age = models.IntegerField(
        label='What is your age?',
```

```
        min=18,
        max=99,
    )
    gender = models.StringField(
        choices=[['Male', 'Male'], ['Female', 'Female'],
                 ['Non-Binary', 'Non-Binary'],
                 ['NoState', 'Prefer not to state']],
        label='What is your gender?',
        widget=widgets.RadioSelect,
    )
    race = models.StringField(
        choices=[
            ['Black', 'African descent, Black, African-American'],
            ['Asian', 'Asian descent, Asian-American'],
            ['Indigenous', 'Indigenous, First Nations, Native American'],
            ['MiddleEastern', 'Middle Eastern or North African'],
            ['Hawaiian', 'Native Hawaiian or Pacific Islander'],
            ['White', 'White'],
            ['Other', 'Other'],
            ['Declined', 'Decline to state']
        ],
        label='What is your race?',
        widget=widgets.RadioSelect,
    )
    ethnicity = models.BooleanField(
        choices=[[True, 'Yes'], [False, 'No']],
        label='Are you of Latinx/o/a or Hispanic origin?',
        widget=widgets.RadioSelect,
    )
    school = models.StringField(
        choices=[
            ['Social', 'School of Social Sciences'],
            ['Ecology', 'School of Social Ecology'],
            ['Law', 'School of Law'],
            ['Humanities', 'School of Humanities'],
            ['Education', 'School of Education'],
            ['Arts', 'Claire Trevor School of the Arts'],
            ['Business', 'The Paul Merage School of Business'],
            ['Biological', 'School of Biological Sciences'],
            ['Engineering', 'The Henry Samueli School of Engineering'],
            ['Computers', 'Donald Bren School of Information and Computer Sciences'],
            ['Interdisciplinary', 'Interdisciplinary Studies'],
```

```
            ['Physical', 'School of Physical Sciences'],
            ['Medicine', 'School of Medicine'],
            ['Nursing', 'Sue and Bill Gross School of Nursing'],
            ['Pharmaceutical', 'School of Pharmacy and Pharmaceutical Sciences'],
            ['Health', 'Program in Public Health']
        ],
        label='Choose the most appropriate option for your school below.',
        widget=widgets.RadioSelect,
    )
    field_of_study = models.StringField(
        label='What is your major or field of study?',
    )
    major_STEM = models.BooleanField(
        choices=[[True, 'Yes'], [False, 'No']],
        label='Are you majoring in a STEM field?',
        widget=widgets.RadioSelect,
    )
    math_courses = models.IntegerField(
        label='How many Mathematics/Statistics courses have you taken since High School?',
        min=0,
        max=99,
    )
    english_is_first_language = models.BooleanField(
        choices=[[True, 'Yes'], [False, 'No']],
        label='Is English your first language?',
        widget=widgets.RadioSelect,
    )
    first_language = models.StringField(
        label='Please state your first language.',
        blank=True,
    )

    # ~~~ Cognitive Reflection Task

    crt_q1 = models.StringField(
        label="If John can drink one barrel of water in 6 days, and Mary can drink one barrel
    )
    crt_q2 = models.StringField(
        label="A man buys a pig for \$60, sells it for \$70, buys it back for \$80, and sells
    )
    crt_q3 = models.StringField(
        label="Jerry received both the 15th highest and the 15th lowest mark in the class. He
```

```
    )


# ~~~~~~~~~~~~~~~~~ PAGES

class Questionnaire_CRT(Page):
    form_model = 'player'
    form_fields = ['crt_q1', 'crt_q2', 'crt_q3']

    @staticmethod
    def is_displayed(player):
        return player.round_number == C.NUM_ROUNDS


class Demographics(Page):
    form_model = 'player'
    form_fields = ['age', 'gender', 'race', 'ethnicity', 'school',
                   'field_of_study', 'major_STEM', 'math_courses',
                   'english_is_first_language', 'first_language']


page_sequence = [Questionnaire_CRT, Demographics]
```

Now let us go through every part of this file.

The import statement

```
from otree.api import *
```

This line is the same in every single oTree app. It imports all of oTree's built-in classes, field types, widgets, and functions. You will never change this line.

The `doc` string

```
doc = """
Standard, end-of-session survey.
Includes Cognitive Reflection Task and demographic survey.
"""
```

The `doc` variable is an optional description of the app. It appears in the admin interface when you click on the app sequence for a session. It has no effect on how the app runs — it is purely informational. Write something useful here so that anyone looking at the admin interface understands what this app does without having to open the code.

Class `C`: Constants

```
class C(BaseConstants):
    NAME_IN_URL = 'survey_experiment'
    PLAYERS_PER_GROUP = None
    NUM_ROUNDS = 1
```

The `C` class holds all the fixed parameters of your app — values that are set before the session starts and do not change while it runs.

Constant	Value	What it means
`NAME_IN_URL`	`'survey_experiment'`	The string that appears in the participant's browser URL during the experiment
`PLAYERS_PER_GROUP`	`None`	Participants are not grouped — this is an individual task
`NUM_ROUNDS`	`1`	Participants go through the page sequence once

> ⚠ Participants can see `NAME_IN_URL`
>
> Whatever string you assign to `NAME_IN_URL` will be visible in the browser address bar during the experiment. Notice that this app uses `'survey_experiment'` rather than `'demographic_survey'` — this is deliberate. The name does not reveal what kind of survey is being administered. Do not use names that hint at the purpose or design of

your study.

Python naming conventions

Constant variables like `NAME_IN_URL` are written in `UPPERCASE_WITH_UNDERSCORES`. Non-constant variables and function names use `lowercase_with_underscores`. Class names use `PascalCase` (e.g., `BasePlayer`, `WaitPage`). VS Code's colour-coding works better when you follow these conventions.

Data model classes: `Subsession`, `Group`, and `Player`

Every `__init__.py` must define three data model classes. These represent the three levels of oTree's data structure and correspond directly to tables in the database.

```
class Subsession(BaseSubsession):
    pass

class Group(BaseGroup):
    pass
```

Both `Subsession` and `Group` contain only `pass` here because this app collects individual participant data only. No group-level or round-level variables are needed.

What `pass` means

In Python, `pass` is a placeholder that means "this class exists but has nothing extra defined in it." oTree requires all three model classes to be present in every `__init__.py`. If you are not adding any fields or methods to `Subsession` or `Group`, you must still include the class definition with `pass` underneath it. Removing either class entirely will crash the app.

The `Player` class and model fields

The `Player` class is where all the participant-level data lives. Every field you define here becomes a column in your exported data.

Field types used in this app

Field type	Used for	Example in this app
`models.IntegerField()`	Whole numbers	`age`, `math_courses`
`models.StringField()`	Text or multiple choice	`gender`, `race`, `school`, `field_of_study`
`models.BooleanField()`	True/False answers	`ethnicity`, `major_STEM`, `english_is_first_language`

Field arguments used in this app

Argument	What it does
`label`	The question or prompt shown to the participant on the page
`min` / `max`	Sets the minimum and maximum values for numeric fields
`choices`	A list of options for multiple choice questions
`widget`	Controls how the input is displayed — `widgets.RadioSelect` shows radio buttons
`blank`	If `True`, the field is optional and participants can skip it

How `choices` works

This is one of the most important things to understand about oTree model fields. The `choices` argument takes a list of two-item lists, where each inner list follows this pattern:

```
choices = [[recorded_in_database, shown_to_participant], ...]
```

Look at the `gender` field:

```
gender = models.StringField(
    choices=[['Male', 'Male'], ['Female', 'Female'],
             ['Non-Binary', 'Non-Binary'],
             ['NoState', 'Prefer not to state']],
    label='What is your gender?',
    widget=widgets.RadioSelect,
)
```

The participant sees four options on screen: Male, Female, Non-Binary, and Prefer not to state. But notice that `'NoState'` is what gets recorded in the database — not `'Prefer not to state'`. The first value in each pair is what oTree saves; the second is what the participant reads.

Now look at the `ethnicity` field:

```
ethnicity = models.BooleanField(
    choices=[[True, 'Yes'], [False, 'No']],
    label='Are you of Latinx/o/a or Hispanic origin?',
    widget=widgets.RadioSelect,
)
```

The participant sees Yes or No. The database records `True` or `False`. This is a clean and efficient way to store binary responses — it makes analysis much easier than storing the strings "Yes" and "No" directly.

> ❗ The `choices` format is always `[recorded, displayed]`
>
> The first value in each pair is stored in the database. The second value is what the participant sees. Never mix these up. If you do, your data will contain the wrong values and you will not catch the error until analysis.

The `blank=True` argument

```
first_language = models.StringField(
    label='Please state your first language.',
    blank=True,
)
```

By default, all form fields in oTree are required — participants cannot proceed to the next page without filling them in. Setting `blank=True` makes a field optional. Here `first_language` is optional because only participants who answered No to `english_is_first_language` need to complete it.

Section comments

```
# ~~~ Cognitive Reflection Task
```

This is just a Python comment — a line beginning with `#` that oTree ignores entirely. It is there purely for readability, separating the demographic fields from the CRT fields. Use comments freely in your own code to mark where one section ends and another begins.

Page classes

Each page in your app is defined as a class that inherits from `Page`. This app has two pages.

`Questionnaire_CRT`

```
class Questionnaire_CRT(Page):
    form_model = 'player'
    form_fields = ['crt_q1', 'crt_q2', 'crt_q3']

    @staticmethod
    def is_displayed(player):
        return player.round_number == C.NUM_ROUNDS
```

Attribute or method	What it does
`form_model = 'player'`	Tells oTree the form fields belong to the `Player` model
`form_fields`	Lists the three CRT fields that will appear as inputs on this page
`@staticmethod`	Required decorator before any Page-level method when working in VS Code or any IDE
`is_displayed`	A built-in oTree method that controls whether this page is shown to a participant. It returns `True` or `False`.

The `is_displayed` method here returns:

```
return player.round_number == C.NUM_ROUNDS
```

Since `C.NUM_ROUNDS = 1`, this evaluates to `player.round_number == 1`, which is always `True`

for a single-round app. In a multi-round app, this would restrict the page to only the final round. For now it is effectively always displayed.

> **`@staticmethod` is required for Page methods in VS Code**
>
> When you define any built-in oTree method inside a Page class — such as `is_displayed`, `vars_for_template`, or `before_next_page` — you must precede it with `@staticmethod`. This is because these methods are defined at the Page level rather than outside it. If you forget `@staticmethod`, the method will either not work or will produce an error.

`Demographics`

```
class Demographics(Page):
    form_model = 'player'
    form_fields = ['age', 'gender', 'race', 'ethnicity', 'school',
                   'field_of_study', 'major_STEM', 'math_courses',
                   'english_is_first_language', 'first_language']
```

This page has no special methods — just `form_model` and `form_fields`. All ten demographic fields will appear on this page in the order they are listed in `form_fields`. There is no `is_displayed` method, so this page is always shown.

The `page_sequence` list

```
page_sequence = [Questionnaire_CRT, Demographics]
```

This list controls the order in which participants see the pages. Participants will complete the CRT first, then the demographics questionnaire. If you swap the order here, they will see the demographics first.

> **Every page must be in `page_sequence`**
>
> If you define a Page class but forget to add it to `page_sequence`, oTree will not show it to participants. Conversely, if you list a page in `page_sequence` that does not have a corresponding class definition, the app will crash when you try to run it.

Summary: The anatomy of `__init__.py`

Section	What it contains	Required?
`from otree.api import *`	The oTree import statement	Always
`doc`	A short description of the app	Optional but recommended
`class C(BaseConstants)`	`NAME_IN_URL`, `PLAYERS_PER_GROUP`, `NUM_ROUNDS`	Always
`class Subsession`	Round-level logic and fields	Always — use `pass` if empty
`class Group`	Group-level logic and fields	Always — use `pass` if empty
`class Player`	Individual participant fields	Always — add your data fields here
Page classes	One class per page, with `form_model`, `form_fields`, and any methods	Always — at least one
`page_sequence`	The ordered list of page classes	Always

Session 2

From the Prisoner's Dilemma to the Stag Hunt

In Session 1, you learned how to set up an oTree project, navigate the folder structure, understand `settings.py`, and read through a simple survey app. Now it is time to work with something more interesting — a two-player game.

oTree comes with several sample games built in, including a one-shot Prisoner's Dilemma. In this session we will use that sample app as our starting point. We will first understand how it works — both the Python logic and the HTML templates — and then begin modifying it to build toward the Stag Hunt game you will submit for your assignment.

This session covers:

- **Inside the Prisoner's Dilemma: `__init__.py`** — Understanding the Python code that powers a two-player game: constants, groups, player decisions, payoff matrices, wait pages, and results.
- **Inside the Prisoner's Dilemma: HTML Templates** — Understanding the four template files that control what participants see: page structure, payoff tables, conditional text, included templates, and the chat widget.
- **Hands-On: Getting started with Assignment 2** — Your first modifications to the Prisoner's Dilemma app: turning a one-shot game into a repeated game and building a history table that tracks every round.

The final chapter contains a quick reference for the oTree features and Python patterns you will need for your Stag Hunt assignment.

Inside the Prisoner's Dilemma: `__init__.py`

When you created your oTree project with sample games in Session 1, one of the apps included was `prisoner` — a one-shot Prisoner's Dilemma. Two players are paired together, each privately chooses to cooperate or defect, and their payoffs depend on the combination of choices.

This app is a natural stepping stone toward the Stag Hunt. Both are two-player games with a 2×2 payoff matrix, and the code patterns you learn here — groups, roles, payoff calculations, wait pages — carry directly into your assignment.

Open the file `prisoner/__init__.py` in VS Code. Here is the full file:

```
from otree.api import *

doc = """
This is a one-shot "Prisoner's Dilemma". Two players are asked separately
whether they want to cooperate or defect. Their choices directly determine the
payoffs.
"""

class C(BaseConstants):
    NAME_IN_URL = 'prisoner'
    PLAYERS_PER_GROUP = 2
    NUM_ROUNDS = 1
    PAYOFF_A = cu(300)
    PAYOFF_B = cu(200)
    PAYOFF_C = cu(100)
    PAYOFF_D = cu(0)

class Subsession(BaseSubsession):
    pass

class Group(BaseGroup):
    pass

class Player(BasePlayer):
    cooperate = models.BooleanField(
```

```
        choices=[[True, 'Cooperate'], [False, 'Defect']],
        doc="""This player's decision""",
        widget=widgets.RadioSelect,
    )

# FUNCTIONS
def set_payoffs(group: Group):
    for p in group.get_players():
        set_payoff(p)

def other_player(player: Player):
    return player.get_others_in_group()[0]

def set_payoff(player: Player):
    payoff_matrix = {
        (False, True): C.PAYOFF_A,
        (True, True): C.PAYOFF_B,
        (False, False): C.PAYOFF_C,
        (True, False): C.PAYOFF_D,
    }
    other = other_player(player)
    player.payoff = payoff_matrix[(player.cooperate, other.cooperate)]

# PAGES
class Introduction(Page):
    timeout_seconds = 100

class Decision(Page):
    form_model = 'player'
    form_fields = ['cooperate']

class ResultsWaitPage(WaitPage):
    after_all_players_arrive = set_payoffs

class Results(Page):
    @staticmethod
    def vars_for_template(player: Player):
        opponent = other_player(player)
        return dict(
            opponent=opponent,
            same_choice=player.cooperate == opponent.cooperate,
            my_decision=player.field_display('cooperate'),
```

```
            opponent_decision=opponent.field_display('cooperate'),
        )

page_sequence = [Introduction, Decision, ResultsWaitPage, Results]
```

Now let us go through each section. Many of these concepts were introduced briefly in Session 1 — here you will see them working together in a real game.

The import and `doc`

```
from otree.api import *

doc = """
This is a one-shot "Prisoner's Dilemma". Two players are asked separately
whether they want to cooperate or defect. Their choices directly determine the
payoffs.
"""
```

These are the same as in the demographic survey app. The import brings in everything oTree needs. The `doc` string is an optional description that appears in the admin interface.

Class `C`: constants for a two-player game

```
class C(BaseConstants):
    NAME_IN_URL = 'prisoner'
    PLAYERS_PER_GROUP = 2
    NUM_ROUNDS = 1
    PAYOFF_A = cu(300)
    PAYOFF_B = cu(200)
    PAYOFF_C = cu(100)
    PAYOFF_D = cu(0)
```

Compare this to the demographic survey's constants:

Constant	Demographic survey	Prisoner's Dilemma
NAME_IN_URL	'survey_experiment'	'prisoner'
PLAYERS_PER_GROUP	None	2
NUM_ROUNDS	1	1

The key difference is `PLAYERS_PER_GROUP = 2`. This tells oTree to automatically divide participants into **pairs**. If 10 participants join a session, oTree creates 5 groups of 2.

Payoff constants with `cu()`

```
PAYOFF_A = cu(300)
PAYOFF_B = cu(200)
PAYOFF_C = cu(100)
PAYOFF_D = cu(0)
```

The `cu()` function stands for **currency unit**. It creates a currency value that oTree knows how to format and display — with dollar signs, decimal places, and proper alignment. Using `cu()` instead of plain numbers ensures consistent formatting throughout the experiment.

These four constants represent the four possible payoffs in the game. By defining them as constants, you can change the payoff structure by editing one line in the `C` class instead of searching through the entire file.

> Why use named constants instead of raw numbers?
>
> Consider two versions of the same code:
>
> ```
> # Hard to understand
> player.payoff = cu(300)
>
> # Easy to understand
> player.payoff = C.PAYOFF_A
> ```
>
> When you return to your code a week later, `C.PAYOFF_A` immediately tells you this is "payoff A from the game matrix." The number `300` tells you nothing about its role in the experiment.

How the payoff labels map to the game

The Prisoner's Dilemma payoff matrix looks like this (your payoff listed first in each cell):

	Partner cooperates	Partner defects
You cooperate	200, 200	0, 300
You defect	300, 0	100, 100

The constants map to this matrix as follows:

Constant	Value	Meaning
`PAYOFF_A`	`cu(300)`	You defect, partner cooperates (temptation)
`PAYOFF_B`	`cu(200)`	Both cooperate (mutual cooperation)
`PAYOFF_C`	`cu(100)`	Both defect (mutual defection)
`PAYOFF_D`	`cu(0)`	You cooperate, partner defects (sucker's payoff)

The data model classes

```
class Subsession(BaseSubsession):
    pass

class Group(BaseGroup):
    pass

class Player(BasePlayer):
    cooperate = models.BooleanField(
        choices=[[True, 'Cooperate'], [False, 'Defect']],
        doc="""This player's decision""",
        widget=widgets.RadioSelect,
    )
```

`Subsession` and `Group`

Both are `pass` — no custom fields needed. The `Subsession` class has no extra logic because there is only one round and no special setup. The `Group` class has no fields because no group-level data is tracked (we only care about individual decisions and payoffs).

Remember from Session 1: these classes must always be present in `__init__.py`, even if they contain nothing. Removing them will crash the app.

The `Player` class

The `Player` class defines one field: `cooperate`. This is the participant's decision in the game.

```
cooperate = models.BooleanField(
    choices=[[True, 'Cooperate'], [False, 'Defect']],
    doc="""This player's decision""",
    widget=widgets.RadioSelect,
)
```

This should look familiar from the demographic survey. Let us review the key parts:

Argument	Value	What it does
`models.BooleanField`	—	Stores `True` or `False`
`choices`	`[[True, 'Cooperate'], [False, 'Defect']]`	Maps database values to display labels
`doc`	`"This player's decision"`	Description in data export (optional)
`widget`	`widgets.RadioSelect`	Displays radio buttons instead of a dropdown

The `choices` argument follows the `[recorded, displayed]` pattern from Session 1. The database stores `True` or `False`. The participant sees "Cooperate" or "Defect."

The functions: payoff logic

The Prisoner's Dilemma app has three functions that handle payoff calculations. These functions sit outside of any class — they are **module-level functions** that oTree calls when needed.

`other_player`: finding the partner

```
def other_player(player: Player):
    return player.get_others_in_group()[0]
```

This function takes a player and returns their partner — the other player in the same group. Here is how it works:

- `player.get_others_in_group()` is a built-in oTree method that returns a **list** of all other players in the same group.
- Since `PLAYERS_PER_GROUP = 2`, there is exactly one other player, so the list has exactly one element.
- `[0]` extracts the first (and only) element from the list.

You will use this function every time you need to look at what the partner chose or what the partner earned.

> **Python lists and indexing**
>
> A list is an ordered collection of items. You access items by their **index** — their position in the list. Python uses **zero-based indexing**, which means the first item is at position `0`, the second at position `1`, and so on.
>
> ```
> my_list = ['apple', 'banana', 'cherry']
> my_list[0]   # returns 'apple'
> my_list[1]   # returns 'banana'
> my_list[2]   # returns 'cherry'
> ```
>
> So `get_others_in_group()[0]` means "get the list of other players, then grab the first one."

`set_payoff`: the payoff matrix as a dictionary

```
def set_payoff(player: Player):
    payoff_matrix = {
        (False, True): C.PAYOFF_A,
        (True, True): C.PAYOFF_B,
        (False, False): C.PAYOFF_C,
```

```
        (True, False): C.PAYOFF_D,
    }
    other = other_player(player)
    player.payoff = payoff_matrix[(player.cooperate, other.cooperate)]
```

This is the heart of the game. It calculates the player's payoff based on both players' decisions. Let us break it down.

The payoff matrix dictionary:

A Python **dictionary** maps keys to values. Here, each key is a **tuple** — a pair of values in parentheses — representing `(my_choice, partner_choice)`. Each value is the corresponding payoff.

Key	My choice	Partner's choice	Payoff
`(True, True)`	Cooperate	Cooperate	`cu(200)`
`(True, False)`	Cooperate	Defect	`cu(0)`
`(False, True)`	Defect	Cooperate	`cu(300)`
`(False, False)`	Defect	Defect	`cu(100)`

The lookup:

```
other = other_player(player)
player.payoff = payoff_matrix[(player.cooperate, other.cooperate)]
```

First, the code gets the partner using `other_player`. Then it constructs a tuple from both players' choices — `(player.cooperate, other.cooperate)` — and uses it as a key to look up the payoff in the dictionary.

For example, if this player cooperated (`True`) and the partner defected (`False`), the key is `(True, False)`, and the payoff is `C.PAYOFF_D = cu(0)`.

The result is stored in `player.payoff`, which is a **built-in oTree field** that tracks each player's earnings. Unlike the custom fields you define yourself, `player.payoff` exists automatically in every app.

> **Python dictionaries**
>
> A dictionary is created with curly braces and contains key-value pairs separated by colons:

```
my_dict = {
    'apple': 3,
    'banana': 5,
}
```

You look up a value by its key: `my_dict['apple']` returns `3`.
In the payoff matrix, the keys are tuples like `(True, False)` instead of strings. This is a common and powerful Python pattern — it lets you look up a value based on a combination of conditions in one clean step, instead of writing a long chain of if/else statements.

The dictionary approach vs. if/else

Without the dictionary, you would need to write:

```
if player.cooperate == True and other.cooperate == True:
    player.payoff = C.PAYOFF_B
elif player.cooperate == True and other.cooperate == False:
    player.payoff = C.PAYOFF_D
elif player.cooperate == False and other.cooperate == True:
    player.payoff = C.PAYOFF_A
else:
    player.payoff = C.PAYOFF_C
```

This works, but the dictionary version is more compact and easier to verify against the game matrix. Both approaches produce the same result — use whichever you find clearer.

`set_payoffs`: applying to the whole group

```
def set_payoffs(group: Group):
    for p in group.get_players():
        set_payoff(p)
```

This function loops through every player in the group and calls `set_payoff` on each one. Since `PLAYERS_PER_GROUP = 2`, the loop runs exactly twice — once for each player.

> **Python `for` loops**
>
> A `for` loop repeats a block of code once for each item in a list:
>
> ```
> for item in my_list:
>  # do something with item
> ```
>
> Here, `group.get_players()` returns a list of Player objects. The variable `p` takes on a different Player object each time through the loop. Indentation defines what code is "inside" the loop — everything indented under the `for` line runs once per item.

The pages

The Prisoner's Dilemma has four pages. Let us look at each one.

`Introduction`: a simple instruction page

```
class Introduction(Page):
    timeout_seconds = 100
```

This page displays the experiment instructions. It has no form fields — participants just read and click "Next." The `timeout_seconds = 100` automatically advances the page after 100 seconds if the participant has not clicked "Next" themselves.

The actual text shown on this page comes from the HTML template `Introduction.html`, which we will look at in the next part.

`Decision`: collecting the player's choice

```
class Decision(Page):
    form_model = 'player'
    form_fields = ['cooperate']
```

This page collects the participant's decision. The two attributes tell oTree:

- `form_model = 'player'` — The form data belongs to the `Player` model.

- `form_fields = ['cooperate']` — Display the `cooperate` field as an input on this page.

When the participant submits the page, their choice (`True` for cooperate, `False` for defect) is saved to `player.cooperate`.

`ResultsWaitPage`: synchronising both players

```
class ResultsWaitPage(WaitPage):
    after_all_players_arrive = set_payoffs
```

This is the first `WaitPage` you are seeing. A wait page is fundamentally different from a regular page:

- A regular `Page` shows content to one player and lets them proceed independently.
- A `WaitPage` **pauses** each player until **all players in the group** have arrived.

This is essential in the Prisoner's Dilemma. You cannot calculate payoffs until both players have submitted their decisions. When Player 1 finishes the Decision page, they see the wait page. When Player 2 also finishes, the wait page releases both players.

`after_all_players_arrive = set_payoffs` tells oTree: "Once both players have arrived at this wait page, run the `set_payoffs` function." This calculates both players' payoffs before they see the results.

> **! `after_all_players_arrive` runs once per group**
>
> This function runs **once** after all players in the group have reached the wait page — not once per player. That is why `set_payoffs` takes a `group` as its argument and loops through all players in the group internally.

> **⚠ Testing wait pages**
>
> When testing locally with `otree devserver`, you need to open **two browser tabs** (or two browsers) to simulate two players in the same group. If you only have one tab open, you will be stuck on the wait page forever because the second player never arrives.

`Results`: showing the outcome

```python
class Results(Page):
    @staticmethod
    def vars_for_template(player: Player):
        opponent = other_player(player)
        return dict(
            opponent=opponent,
            same_choice=player.cooperate == opponent.cooperate,
            my_decision=player.field_display('cooperate'),
            opponent_decision=opponent.field_display('cooperate'),
        )
```

This page displays the round's results. It uses `vars_for_template` to pass several calculated variables to the HTML template.

`vars_for_template` is a built-in oTree method that lets you send custom data from Python to the HTML template. Whatever keys you include in the returned dictionary become available in the template using `{ variable_name }` syntax.

Let us look at each variable:

Variable	Value	Purpose
`opponent`	The other player object	Access opponent's data in the template
`same_choice`	`True` if both made the same choice	Used for conditional display
`my_decision`	`'Cooperate'` or `'Defect'` (display label)	Human-readable version of the choice
`opponent_decision`	`'Cooperate'` or `'Defect'` (display label)	Human-readable version of partner's choice

`player.field_display('cooperate')` is a built-in oTree method that returns the **display label** for a field's value instead of the raw database value. So instead of `True`, it returns `'Cooperate'`. This is what you want to show participants — they should see words, not `True`/`False`.

`player.cooperate == opponent.cooperate` creates a Boolean value: `True` if both players made the same choice, `False` otherwise. The template uses this to display different messages depending on the outcome.

> **Variables oTree passes automatically**
>
> You do not need `vars_for_template` for everything. oTree automatically makes `player`, `group`, `subsession`, `participant`, `session`, and `C` available in every template. You can always write `{ player.payoff }` or `{ C.NUM_ROUNDS }` without any extra work.
> Use `vars_for_template` for values that require calculation or processing — like the display labels, the opponent's data, or derived Boolean flags.

The page sequence

```
page_sequence = [Introduction, Decision, ResultsWaitPage, Results]
```

Participants move through the pages in this order:

1. **Introduction** — Read the instructions (with a 100-second timer).
2. **Decision** — Choose to cooperate or defect.
3. **ResultsWaitPage** — Wait for the partner. Once both arrive, payoffs are calculated.
4. **Results** — See what both players chose and the resulting payoffs.

Since `NUM_ROUNDS = 1`, this sequence runs exactly once. In the next hands-on session, you will change `NUM_ROUNDS` and see the entire sequence repeat.

Comparing the survey app to the Prisoner's Dilemma

Now that you have seen both a survey app and a game app, here is a side-by-side comparison of the key differences:

Feature	Demographic survey	Prisoner's Dilemma
`PLAYERS_PER_GROUP`	`None` (individual)	`2` (pairs)
`NUM_ROUNDS`	`1`	`1`
`Group` class	`pass`	`pass`
`Player` fields	Many (age, gender, etc.)	One (`cooperate`)
Functions	None	`set_payoff`, `set_payoffs`, `other_player`
`WaitPage`	Not used	Used (to sync both players)
`vars_for_template`	Not used	Used (to pass opponent data)
`form_fields`	Many fields on one page	One field on one page

The game app introduces three concepts not needed in surveys: **groups** (pairing players), **wait pages** (synchronising players), and **payoff functions** (calculating earnings based on joint decisions).

Summary

Concept	What it does	Where it appears
`PLAYERS_PER_GROUP = 2`	Pairs participants into groups of 2	`class C`
`cu()`	Creates a formatted currency value	Payoff constants
`models.BooleanField` with `choices`	Stores True/False, displays custom labels	`Player` class
`get_others_in_group()[0]`	Returns the other player in the group	`other_player` function
Dictionary with tuple keys	Maps `(my_choice, partner_choice)` to payoff	`set_payoff` function
`for` loop over `group.get_players()`	Runs code for each player in the group	`set_payoffs` function
`WaitPage`	Pauses until all group members arrive	`ResultsWaitPage`
`after_all_players_arrive`	Runs a function once all players reach the wait page	`ResultsWaitPage`
`vars_for_template`	Passes custom variables to the HTML template	`Results` page
`field_display('field')`	Returns the display label instead of the raw value	`Results` page
`player.payoff`	Built-in field for storing a player's earnings	Set in `set_payoff`

Inside the Prisoner's Dilemma: HTML Templates

Every page class in your `__init__.py` has a matching HTML template file that controls what participants see on screen. The Python code handles the logic — grouping, payoffs, page flow. The HTML templates handle the presentation — text, tables, buttons, and layout.

The Prisoner's Dilemma app has four template files:

File	Page class	What it shows
`Introduction.html`	`Introduction`	Instructions for the game
`Decision.html`	`Decision`	The payoff matrix and choice buttons
`Results.html`	`Results`	What both players chose and the payoff
`instructions.html`	(none — included in other pages)	A reusable instructions card

Open each of these files in VS Code as we go through them.

The basic template structure

Every oTree HTML template uses the same skeleton:

```
{{ block title }}
  Page Title Goes Here
{{ endblock }}

{{ block content }}
  Everything the participant sees goes here.

  {{ next_button }}
{{ endblock }}
```

There are two blocks:

- **`{ block title }`** — Sets the text shown at the top of the page and in the browser tab.
- **`{ block content }`** — Contains all the page content: text, tables, images, form fields, and the Next button.

The `{ next_button }` tag renders a standard "Next" button that advances the participant to the next page. If you leave it out, participants have no way to proceed (unless you use buttons that submit the form directly, as the Decision page does).

> **!** `{ }` is oTree's template syntax
>
> Everything wrapped in double curly braces `{ }` is processed by oTree before the page is sent to the participant's browser. Plain HTML is rendered directly. You will see `{ }` used to display variables, insert buttons, include other files, and write conditional logic.

`Introduction.html`: a minimal page

```
{{ block title }}Introduction{{ endblock }}
{{ block content }}

    {{ include_sibling 'instructions.html' }}

    {{ next_button }}

{{ endblock }}
```

This is the simplest template in the app. It does three things:

1. Sets the page title to "Introduction."
2. Includes the content of another file (`instructions.html`) using `{ include_sibling }`.
3. Displays the Next button.

`{ include_sibling }`: reusing content across pages

```
{{ include_sibling 'instructions.html' }}
```

This tag inserts the entire contents of `instructions.html` into this page. It is like copy-pasting the file's content, but without actually duplicating it. If you update `instructions.html`, the change automatically appears everywhere it is included.

The word "sibling" means the included file must be in the **same folder** as the template that includes it — in this case, both files are inside the `prisoner/` app folder.

> When to use `include_sibling`
>
> Use `{ include_sibling }` when you want the same block of content to appear on multiple pages. In this app, the instructions card appears on both the Introduction page and the Decision page. Instead of copying the table into both files, you write it once in `instructions.html` and include it wherever you need it.
> This is the same principle as defining constants in `class C` — write it once, use it everywhere, change it in one place.

`instructions.html`: a reusable card

```
<div class="card bg-light m-3">
    <div class="card-body">

    <h3>
        Instructions
    </h3>

    <p>
        In this study, you will be randomly and anonymously paired with another
        participant.
        Each of you simultaneously and privately chooses whether you want to
        cooperate or defect.
        Your payoffs will be determined by the choices of both as below:
    </p>
    <p><i>In each cell, the amount to the left is the payoff for
        you and to the right for the other participant.</i></p>

    <table class='table table-bordered text-center'
           style='width: auto; margin: auto'>
        <tr>
            <th colspan=2 rowspan=2></th>
            <th colspan=2>The Other Participant</th>
```

```
        </tr>
        <tr>
            <th>Cooperate</th>
            <th>Defect</th>
        </tr>
        <tr>
            <th rowspan=2>
                <span style="transform: rotate(-90deg);">You</span>
            </th>
            <th>Cooperate</th>
            <td>{{ C.PAYOFF_B }}, {{ C.PAYOFF_B }}</td>
            <td>0, {{ C.PAYOFF_A }}</td>
        </tr>
        <tr>
            <th>Defect</th>
            <td>{{ C.PAYOFF_A }}, 0</td>
            <td>{{ C.PAYOFF_C }}, {{ C.PAYOFF_C }}</td>
        </tr>
    </table>

</div>
</div>
```

This file is **not** a full page — it has no `{ block title }` or `{ block content }`. It is a fragment designed to be included inside other pages. Let us go through the key elements.

HTML basics: tags and attributes

If you have not worked with HTML before, here is what you need to know:

- HTML uses **tags** to structure content. Tags come in pairs: `<p>` starts a paragraph, `</p>` ends it.
- Tags can have **attributes** that modify their behaviour: `<table class="table">` creates a table with the CSS class `"table"`.
- Common tags: `<p>` (paragraph), `<h3>` (heading), `<table>` (table), `<tr>` (table row), `<th>` (table header cell), `<td>` (table data cell), `<div>` (a container), `<i>` (italic text), `<b>` (bold text), `<span>` (inline container).

The Bootstrap card

```
<div class="card bg-light m-3">
    <div class="card-body">
        ...
    </div>
</div>
```

oTree includes **Bootstrap**, a CSS framework that provides pre-styled components. The classes used here are:

Class	What it does
`card`	Creates a card component — a bordered box with padding
`bg-light`	Sets a light grey background colour
`m-3`	Adds margin (spacing) around the card
`card-body`	The inner content area of the card

You do not need to write any CSS to use these — Bootstrap handles the styling. This is a quick way to make your pages look professional.

The payoff table

```
<table class='table table-bordered text-center'
       style='width: auto; margin: auto'>
```

This creates a table with Bootstrap styling:

Class/Style	What it does
`table`	Applies Bootstrap's default table styling
`table-bordered`	Adds borders around all cells
`text-center`	Centres text in all cells
`width: auto`	Makes the table only as wide as its content
`margin: auto`	Centres the table on the page

Displaying constants in the table

```
<td>{{ C.PAYOFF_B }}, {{ C.PAYOFF_B }}</td>
```

This displays two values from the `C` class, separated by a comma. Since `C.PAYOFF_B = cu(200)`, the cell shows `$200.00, $200.00` (or whatever currency format oTree is configured to use). Remember: `C` is automatically available in every template — you do not need `vars_for_template` to access constants.

Table structure: `colspan` and `rowspan`

```
<th colspan=2 rowspan=2></th>
<th colspan=2>The Other Participant</th>
```

The `colspan` and `rowspan` attributes make a cell span across multiple columns or rows. This is how the table headers are structured:

- `colspan=2 rowspan=2` on the empty top-left cell makes it span 2 columns and 2 rows, creating the blank corner of the matrix.
- `colspan=2` on "The Other Participant" makes it span the two choice columns (Cooperate and Defect).

> Understanding table layout
>
> A 2×2 payoff matrix actually needs more than 2 columns and 2 rows in HTML because of the row and column headers. The full table is structured like this:
>
> | | | Cooperate | Defect |
> |---|---|---|---|
> | | | (header spanning 2 cols) | |
> | **You** | **Cooperate** | Payoff, Payoff | Payoff, Payoff |
> | | **Defect** | Payoff, Payoff | Payoff, Payoff |
>
> The `colspan` and `rowspan` attributes handle the merging of header cells to create this layout.

`Decision.html`: buttons as form submission

```
{{ block title }}Your Choice{{ endblock }}
{{ block content }}

    <div class="form-group required">
        <table class="table table-bordered text-center"
               style="width: auto; margin: auto">
            <tr>
                <th colspan="2" rowspan="2"></th>
                <th colspan="2">The Other Participant</th>
            </tr>
            <tr>
                <th>Cooperate</th>
                <th>Defect</th>
            </tr>
            <tr>
                <th rowspan="2"><span>You</span></th>
                <td>
                    <button name="cooperate" value="True"
                            class="btn btn-primary btn-large">
                        I will cooperate
                    </button>
                </td>
                <td>{{C.PAYOFF_B}}, {{C.PAYOFF_B}}</td>
                <td>{{ C.PAYOFF_D }}, {{C.PAYOFF_A}}</td>
            </tr>
            <tr>
                <td>
                    <button name="cooperate" value="False"
                            class="btn btn-primary btn-large">
                        I will defect
                    </button>
                </td>
                <td>{{C.PAYOFF_A}}, {{ C.PAYOFF_D }}</td>
                <td>{{C.PAYOFF_C}}, {{C.PAYOFF_C}}</td>
            </tr>
        </table>
    </div>

    <p>Here you can chat with the other participant.</p>
```

```
    {{ chat }}

    {{ include_sibling 'instructions.html' }}

{{ endblock }}
```

This is the most complex template in the app. Let us focus on the new elements.

Buttons instead of radio buttons

The `Decision` page class has `form_fields = ['cooperate']`, which means oTree expects the `cooperate` field to be submitted on this page. But instead of using `{ formfields }` to display the standard radio buttons, this template uses **HTML buttons** embedded in the payoff matrix:

```
<button name="cooperate" value="True"
        class="btn btn-primary btn-large">
    I will cooperate
</button>
```

When a participant clicks this button, oTree sets `player.cooperate = True` and submits the page. The key attributes are:

Attribute	Value	What it does
`name`	`"cooperate"`	Must match the field name in `form_fields`
`value`	`"True"`	The value submitted for this field
`class`	`"btn btn-primary btn-large"`	Bootstrap classes for a styled blue button

This approach integrates the form submission directly into the payoff matrix, so participants can make their choice by clicking a button in the relevant row of the table. There is no separate Next button — clicking either button submits the form and advances the page.

> **{ formfields } vs. custom buttons**
>
> You have two options for collecting form data:
>
> 1. **`{ formfields }`** — Automatically renders all fields in `form_fields` as standard inputs (radio buttons, text fields, etc.). Simple and reliable.
> 2. **Custom HTML buttons** — You write the buttons yourself using `<button name="..." value="...">`. This gives you full control over layout and appearance.
>
> The custom approach is more work but lets you place inputs exactly where you want them — for example, inside a payoff matrix.

The `{ chat }` widget

```
{{ chat }}
```

This single tag adds oTree's built-in chat widget to the page. Participants in the same group can send messages to each other in real time. This is included in the sample app to demonstrate the feature, but you would typically only use it if your experimental design requires communication between participants.

> **You probably will not use `{ chat }` in your assignment**
>
> The Stag Hunt assignment does not require a chat feature. When you build your own app, you can simply leave this out. It is included here because the Prisoner's Dilemma sample is designed to showcase oTree's capabilities.

`Results.html`: conditional display

```
{{ block title }}Results{{ endblock }}
{{ block content }}

    <p>
        {{ if same_choice }}
            Both of you chose to {{ my_decision }}.
        {{ else }}
            You chose to {{ my_decision }} and the other participant
```

```
            chose to {{ opponent_decision }}.
        {{ endif }}
    </p>

    <p>
        As a result, you earned {{ player.payoff }}.
    </p>

    {{ next_button }}

    {{ include_sibling 'instructions.html' }}

{{ endblock }}
```

This template introduces two important template features.

`{ if }` / `{ else }` / `{ endif }`: conditional display

```
{{ if same_choice }}
    Both of you chose to {{ my_decision }}.
{{ else }}
    You chose to {{ my_decision }} and the other participant
    chose to {{ opponent_decision }}.
{{ endif }}
```

This is the template equivalent of a Python if/else statement. It controls which text is shown based on a condition:

- If `same_choice` is `True` (both players made the same decision), the first paragraph appears.
- If `same_choice` is `False`, the second paragraph appears instead.

The variable `same_choice` was created in `vars_for_template` in the `Results` page class. Any Boolean variable can be used as a condition in `{ if }`.

> **Template `{ if }` syntax**
>
> The full syntax supports several patterns:

```
{{/* Simple condition */}}
{{ if variable }}
    Shown when variable is True (or non-empty)
{{ endif }}

{{/* If/else */}}
{{ if variable }}
    Shown when True
{{ else }}
    Shown when False
{{ endif }}

{{/* Comparisons */}}
{{ if player.round_number == 1 }}
    This is the first round.
{{ endif }}

{{ if player.payoff > 0 }}
    You earned something!
{{ endif }}
```

Always close your `{ if }` blocks with `{ endif }`. Forgetting `{ endif }` will cause a template error.

Displaying variables from `vars_for_template`

```
{{ my_decision }}
{{ opponent_decision }}
{{ player.payoff }}
```

Variables created in `vars_for_template` (like `my_decision` and `opponent_decision`) are accessed by name. Built-in variables (like `player.payoff`) are accessed using dot notation. Both use the `{ }` syntax.

File structure recap

Here is how the four files relate to each other:

```
prisoner/
   __init__.py           ← Python logic (constants, payoffs, pages)
   Introduction.html     ← Instructions page (includes instructions.html)
   Decision.html         ← Choice page with buttons (includes instructions.html)
   Results.html          ← Outcome page (includes instructions.html)
   instructions.html     ← Reusable instructions card (included by others)
```

The `__init__.py` file defines the **logic** — what data to collect, how to calculate payoffs, and what variables to pass to templates. The HTML files define the **presentation** — what text participants read, how the payoff matrix looks, and where the buttons go.

Summary

Concept	Syntax	What it does
`{ block title }`	Template block	Sets the page title
`{ block content }`	Template block	Contains all page content
`{ next_button }`	Template tag	Renders the "Next" button
`{ variable_name }`	Template variable	Displays a variable's value
`{ C.CONSTANT }`	Template variable	Displays a constant from the `C` class
`{ player.field }`	Template variable	Displays a player field's value
`{ if } ... {{ endif }}`	Conditional	Shows content based on a condition
`{ if } ... {{ else }} ... {{ endif }}`	Conditional	Shows one of two blocks
`{ include_sibling 'file.html' }`	Include	Inserts content from another file
`{ formfields }`	Form rendering	Displays all form fields automatically
`{ chat }`	Chat widget	Adds a real-time group chat
`<button name="field" value="val">`	HTML button	Submits a form field with a specific value
Bootstrap classes (`table`, `card`, `btn`)	CSS classes	Pre-built styling from Bootstrap
`colspan` / `rowspan`	HTML attributes	Makes table cells span multiple columns/rows

Hands-On: Getting started with Assignment 2

Now that you understand how the Prisoner's Dilemma app works — both the Python logic and the HTML templates — it is time to start modifying it. In this part you will make several changes that move the app closer to the Stag Hunt assignment:

1. **Turn the one-shot game into a repeated game** by increasing the number of rounds and keeping groups fixed.
2. **Add a game history table** so that participants can see what happened in every previous round.
3. **Add roles** so that each player has a distinct identity within the group.
4. **Add a timer with a default choice** so that the game does not stall if a participant is slow.
5. **Add visual elements** like images and styled text to improve the participant experience.

By the end of this part, you will have a working multi-round Prisoner's Dilemma with several features from the assignment — and you will understand the patterns needed to build the rest on your own.

> **!** Before you start
>
> Make a copy of your `prisoner` app folder before making any changes. This way you can always go back to the original if something breaks. In VS Code, right-click the `prisoner` folder in the Explorer panel and choose **Copy**, then **Paste**. Rename the copy something like `stag_hunt`.
> After renaming, open `stag_hunt/__init__.py` and change `NAME_IN_URL` to something different from `'prisoner'` — for example, `'stag_hunt'`. Each app must have a unique `NAME_IN_URL`.

Step 0: register the new app in `settings.py`

Before you can demo your new `stag_hunt` app, you need to tell oTree it exists. Open `settings.py` and add a new session dictionary to your `SESSION_CONFIGS` list:

```
SESSION_CONFIGS = [
    # ... your existing session dictionaries ...

    dict(
        name='stag_hunt',
        display_name="Stag Hunt (in progress)",
        app_sequence=['stag_hunt'],
        num_demo_participants=2,
    ),
]
```

Notice that `num_demo_participants=2` matches `PLAYERS_PER_GROUP = 2` in the app. If you set this to 1, the demo will get stuck on the wait page because the second player never arrives.

> **❗ Every app must be registered in `settings.py`**
>
> Adding the `stag_hunt` folder to your project is not enough on its own. oTree only runs apps that appear in `SESSION_CONFIGS`. If you skip this step and try to demo the app, it will not show up on the demo page.

Step 1: change `NUM_ROUNDS` to make this a repeated game

Now open `stag_hunt/__init__.py` and find the constants:

```
class C(BaseConstants):
    NAME_IN_URL = 'stag_hunt'
    PLAYERS_PER_GROUP = 2
    NUM_ROUNDS = 1
    PAYOFF_A = cu(300)
    PAYOFF_B = cu(200)
    PAYOFF_C = cu(100)
    PAYOFF_D = cu(0)
```

Change `NUM_ROUNDS` from `1` to `5`:

```
NUM_ROUNDS = 5
```

Save both files. The change to `settings.py` registers the app so it appears on the demo page. The change to `NUM_ROUNDS` tells oTree to repeat the entire page sequence 5 times — every page in `page_sequence` will now appear in every round.

> ⚠ Reset the database after editing `settings.py` or changing model fields
>
> Whenever you edit `settings.py` or change constants like `NUM_ROUNDS`, you must reset the database before running the server:
>
> ```
> otree resetdb
> ```
>
> Confirm with `y` when prompted. If you skip this step, you will get database errors when you try to run the app.

To verify everything is working, start the server with `otree devserver`, open your browser to `http://localhost:8000/demo`, and confirm that "Stag Hunt (in progress)" appears on the demo page. Click on it and open **two browser tabs** to simulate both players. You should see the original Prisoner's Dilemma flow — but now repeating for 5 rounds. Press **Ctrl + C** in the terminal to stop the server when you are done.

Step 2: show the introduction only in round 1

With `NUM_ROUNDS = 5`, the Introduction page currently appears in every round — participants would re-read the instructions five times. You need to restrict it to round 1 only.

Add an `is_displayed` method to the `Introduction` page class:

```
class Introduction(Page):
    timeout_seconds = 100

    @staticmethod
    def is_displayed(player):
        return player.round_number == 1
```

`is_displayed` returns `True` or `False`. If it returns `True`, the page appears. If `False`, the page is skipped for that round. Here, `player.round_number == 1` is `True` only in the first round, so the instructions appear once and are skipped in rounds 2 through 5.

> ℹ `player.round_number`
>
> Every player object has a built-in attribute called `round_number` that tells you the current round. It starts at `1` (not `0`) and goes up to `NUM_ROUNDS`.

You can use `is_displayed` to show any page conditionally. For example, if you wanted a page to appear only in round 4 to announce a rule change, you would write:

```
@staticmethod
def is_displayed(player):
    return player.round_number == 4
```

Step 3: keep groups fixed across rounds

By default, when `NUM_ROUNDS` is greater than 1, oTree keeps the same group assignments across rounds — so you do not need to do anything extra for this step. Your pairs will remain the same from round 1 through round 5.

However, if you wanted to **randomize** groups in round 1 and then keep them fixed, you would add a `creating_session` function:

```
def creating_session(subsession: Subsession):
    if subsession.round_number == 1:
        subsession.group_randomly()
    else:
        subsession.group_like_round(1)
```

This function runs automatically at the start of each round (oTree recognises the name `creating_session` and calls it). In round 1, `group_randomly()` shuffles participants into random pairs. In all later rounds, `group_like_round(1)` copies the group assignments from round 1 so that pairs stay together.

For the Stag Hunt assignment, you will need this — the assignment states that "each subject should retain their role for the duration of the experiment" with roles assigned randomly.

> Add this function now
>
> Even though the default behaviour already keeps groups fixed, adding the `creating_session` function explicitly is good practice. It makes your code self-documenting — anyone reading it can immediately see that groups are randomized in round 1 and fixed thereafter. Place this function after the `Player` class and before the `set_payoffs` function.

Step 4: add a `per_round_payoff` field

Currently the Prisoner's Dilemma stores payoffs in the built-in `player.payoff` field. This works for a one-shot game, but for a repeated game you want to track each round's earnings separately. Add a new field to the `Player` class:

```
class Player(BasePlayer):
    cooperate = models.BooleanField(
        choices=[[True, 'Cooperate'], [False, 'Defect']],
        doc="""This player's decision""",
        widget=widgets.RadioSelect,
    )
    per_round_payoff = models.CurrencyField()
```

Then update the `set_payoff` function to use this new field:

```
def set_payoff(player: Player):
    payoff_matrix = {
        (False, True): C.PAYOFF_A,
        (True, True): C.PAYOFF_B,
        (False, False): C.PAYOFF_C,
        (True, False): C.PAYOFF_D,
    }
    other = other_player(player)
    player.per_round_payoff = payoff_matrix[(player.cooperate, other.cooperate)]
    player.payoff = player.per_round_payoff
```

Now each round's earnings are saved in `per_round_payoff`, and the built-in `player.payoff` is also updated (oTree uses this for the final payment calculation).

Step 5: build the game history in `vars_for_template`

This is the most important new feature. You want to display a table on the Decision page that shows what happened in every previous round — the round number, what you chose, what your partner chose, what you each earned, and your running total.

Add a `vars_for_template` method to the `Decision` page:

```
class Decision(Page):
    form_model = 'player'
    form_fields = ['cooperate']

    @staticmethod
    def vars_for_template(player: Player):
        game_history = []
        cumulative_payoff = 0
```

```
        for p in player.in_previous_rounds():
            other_p = p.get_others_in_group()[0]
            cumulative_payoff += p.per_round_payoff
            game_history.append({
                'round': p.round_number,
                'my_choice': p.field_display('cooperate'),
                'my_payoff': p.per_round_payoff,
                'partner_choice': other_p.field_display('cooperate'),
                'partner_payoff': other_p.per_round_payoff,
                'cumulative': cumulative_payoff,
            })
        return dict(
            game_history=game_history,
        )
```

Let us go through this step by step.

Creating an empty list

```
game_history = []
cumulative_payoff = 0
```

`game_history` starts as an empty list. Each previous round will add one entry (a dictionary) to this list. `cumulative_payoff` is a running total that starts at zero.

Looping through previous rounds

```
for p in player.in_previous_rounds():
```

`player.in_previous_rounds()` is a built-in oTree method that returns a list of the **same player's** objects from all earlier rounds. If you are in round 4, it returns the player objects from rounds 1, 2, and 3. In round 1, it returns an empty list (there are no previous rounds), so the loop does not run and the history table is empty.

Each `p` in the loop is a full Player object for that earlier round — you can access `p.cooperate`, `p.per_round_payoff`, `p.round_number`, and any other field.

Getting the partner for each round

```
other_p = p.get_others_in_group()[0]
```

This gets the partner from that specific round. Since groups are fixed, it will always be the same person — but you still need to use `p.get_others_in_group()` (not `player.get_others_in_group()`) because `p` is the player object from the correct round.

Building each row

```
cumulative_payoff += p.per_round_payoff
game_history.append({
    'round': p.round_number,
    'my_choice': p.field_display('cooperate'),
    'my_payoff': p.per_round_payoff,
    'partner_choice': other_p.field_display('cooperate'),
    'partner_payoff': other_p.per_round_payoff,
    'cumulative': cumulative_payoff,
})
```

The `+=` operator adds `p.per_round_payoff` to the running total. Then `game_history.append(...)` adds a dictionary to the list. Each dictionary represents one row of the history table, with named keys that make the template code easy to read.

`p.field_display('cooperate')` returns the display label (`'Cooperate'` or `'Defect'`) instead of the raw value (`True` or `False`). This is what participants should see.

> **Lists vs. dictionaries for table rows**
>
> In the example above, each row is a **dictionary** with named keys like `'round'`, `'my_choice'`, etc. You could also use a **list** where each element is accessed by position:

```
# Dictionary approach (more readable)
game_history.append({
    'round': p.round_number,
    'my_choice': p.field_display('cooperate'),
})
# In template: {{ row.round }}, {{ row.my_choice }}

# List approach (more compact)
game_history.append([
    p.round_number,
    p.field_display('cooperate'),
])
# In template: {{ row.0 }}, {{ row.1 }}
```

The dictionary approach is recommended because the template code is self-documenting — `{ row.round }` is clearer than `{ row.0 }`.

Step 6: display the history table in the template

Open `Decision.html` and add the history table. Place it above or below the payoff matrix — wherever you think participants will find it most useful. Here is the HTML:

```
{{ if game_history }}
<h4>Game History</h4>
<table class="table table-bordered text-center" style="width: auto; margin: auto">
    <thead>
        <tr>
            <th>Round</th>
            <th>Your Choice</th>
            <th>Your Payoff</th>
            <th>Partner's Choice</th>
            <th>Partner's Payoff</th>
            <th>Cumulative Payoff</th>
        </tr>
    </thead>
    <tbody>
        {{ for row in game_history }}
        <tr>
            <td>{{ row.round }}</td>
            <td>{{ row.my_choice }}</td>
```

```
            <td>{{ row.my_payoff }}</td>
            <td>{{ row.partner_choice }}</td>
            <td>{{ row.partner_payoff }}</td>
            <td>{{ row.cumulative }}</td>
        </tr>
        {{ endfor }}
    </tbody>
</table>
{{ endif }}
```

Let us go through the new template features.

`{ if game_history }`: checking if the list is non-empty

```
{{ if game_history }}
    ...
{{ endif }}
```

In round 1, `game_history` is an empty list. In oTree templates (and Python), an empty list is "falsy" — it evaluates to `False` in a condition. So the entire table is hidden in round 1 (when there is no history to show) and appears from round 2 onward.

`{ for row in game_history }`: looping through rows

```
{{ for row in game_history }}
<tr>
    <td>{{ row.round }}</td>
    <td>{{ row.my_choice }}</td>
    ...
</tr>
{{ endfor }}
```

This is the template equivalent of a Python `for` loop. It repeats the `<tr>...</tr>` block once for each entry in `game_history`. Each time through the loop, `row` takes on the next dictionary from the list, and you access its values with dot notation: `{ row.round }`, `{ row.my_choice }`, etc.

Always close `{ for }` loops with `{ endfor }`.

`<thead>` and `<tbody>`

These are standard HTML tags that separate the header row(s) from the data rows. `<thead>` wraps the column headers. `<tbody>` wraps the data. They have no visual effect by default but help with accessibility and some CSS styling.

Step 7: update the Results page

You should also add the game history to the Results page so that participants can see the full record after each round. Add `vars_for_template` to the `Results` page:

```
class Results(Page):
    @staticmethod
    def vars_for_template(player: Player):
        opponent = other_player(player)
        game_history = []
        cumulative_payoff = 0
        for p in player.in_all_rounds():
            other_p = p.get_others_in_group()[0]
            cumulative_payoff += p.per_round_payoff
            game_history.append({
                'round': p.round_number,
                'my_choice': p.field_display('cooperate'),
                'my_payoff': p.per_round_payoff,
                'partner_choice': other_p.field_display('cooperate'),
                'partner_payoff': other_p.per_round_payoff,
                'cumulative': cumulative_payoff,
            })
        return dict(
            opponent=opponent,
            same_choice=player.cooperate == opponent.cooperate,
            my_decision=player.field_display('cooperate'),
            opponent_decision=opponent.field_display('cooperate'),
            game_history=game_history,
        )
```

! `in_previous_rounds()` vs. `in_all_rounds()`

Notice that the Decision page uses `player.in_previous_rounds()` but the Results page uses `player.in_all_rounds()`. The difference:

- **`in_previous_rounds()`** — Returns player objects from rounds **before** the current round. Use this on the Decision page because the current round has not been played yet.
- **`in_all_rounds()`** — Returns player objects from **all** rounds including the current one. Use this on the Results page because the current round's outcome is now known.

Then add the same history table HTML to `Results.html`, inside the `{ block content }` section.

Step 8: add roles to your game

The Stag Hunt assignment requires two distinct roles — Hunter 1 and Hunter 2 — that are randomly assigned and fixed for the duration of the experiment. Let us add roles to the Prisoner's Dilemma as practice.

Defining role constants

Add role constants to your `C` class:

```
class C(BaseConstants):
    NAME_IN_URL = 'stag_hunt'
    PLAYERS_PER_GROUP = 2
    NUM_ROUNDS = 5
    PAYOFF_A = cu(300)
    PAYOFF_B = cu(200)
    PAYOFF_C = cu(100)
    PAYOFF_D = cu(0)
    PLAYER1_ROLE = 'Player_1'
    PLAYER2_ROLE = 'Player_2'
```

When `PLAYERS_PER_GROUP` is set to a number, oTree automatically assigns each player a position within the group: `id_in_group = 1` for the first player, `id_in_group = 2` for the second. If you define constants ending in `_ROLE` that follow the naming pattern `PLAYER1_ROLE`, `PLAYER2_ROLE`, etc., oTree maps them to these positions automatically.

After adding these, you can use `player.role` anywhere in your code to check which role a player has:

```
if player.role == C.PLAYER1_ROLE:
    # do something specific to Player 1
```

Displaying roles in templates

In any HTML template, you can show the player their role:

```
<p>You are {{ player.role }}.</p>
```

You can also use `{ if }` to display different content based on role:

```
{{ if player.role == C.PLAYER1_ROLE }}
    <p>You are Player 1.</p>
{{ else }}
    <p>You are Player 2.</p>
{{ endif }}
```

Displaying the current round

While we are modifying templates, add the round number so participants know which round they are in. This is useful on both the Decision and Results pages:

```
<p>Round {{ player.round_number }} of {{ C.NUM_ROUNDS }}</p>
```

Both `player.round_number` and `C.NUM_ROUNDS` are automatically available in every template.

Step 9: add a timer with a default choice

The assignment requires a timer on the decision page. If the timer expires, the computer should automatically make a choice for the participant. You already have `timeout_seconds` from the Introduction page, but on a page with form fields you also need to handle what happens when the timer runs out.

Setting the timer

Add `timeout_seconds` to the Decision page:

```
class Decision(Page):
    form_model = 'player'
    form_fields = ['cooperate']
    timeout_seconds = 60
```

A countdown timer will appear on the page. When it reaches zero, oTree automatically submits the page.

Handling the timeout

The problem is that if the timer expires and the participant has not made a choice, the `cooperate` field will be empty — and your payoff calculation will crash. You need `before_next_page` to set a default:

```
class Decision(Page):
    form_model = 'player'
    form_fields = ['cooperate']
    timeout_seconds = 60

    @staticmethod
    def before_next_page(player, timeout_happened):
        if timeout_happened:
            player.cooperate = True
```

`before_next_page` runs after the page is submitted but before the player moves on. The `timeout_happened` parameter is `True` when the timer expired and `False` when the participant submitted normally. Here, if the timer ran out, the code sets `cooperate` to `True` — the computer cooperates on their behalf.

> ⚠ Always handle timeouts on form pages
>
> If a page has both `form_fields` and `timeout_seconds`, you **must** use `before_next_page` to assign a default value. Without it, a timeout will leave the field as `None`, and any code that tries to use that field's value will crash.

Step 10: add images and styled text

The assignment requires visual elements — happy/sad face images on the results page and role-specific text colours. Let us practice both.

Adding images with `{ static }`

To display an image in a template, you need two things: the image file in the right folder, and an `<img>` tag in the template.

Step 1: Place your image files (e.g., `happy.png` and `sad.png`) in the `_static/` folder at the top level of your project — the same level as `settings.py`.

Step 2: Reference them in your template using `{ static }`:

```
{{ if same_choice }}
    <img src="{{ static 'happy.png' }}" alt="Happy face" width="80">
{{ else }}
    <img src="{{ static 'sad.png' }}" alt="Sad face" width="80">
{{ endif }}
```

The `{ static 'filename' }` tag generates the correct URL for files in the `_static/` folder. The `alt` attribute provides text for screen readers (accessibility), and `width` controls the display size in pixels.

> **Where to find free images**
>
> You can find simple emoji-style images for free on sites like OpenMoji or Twemoji. Download the image files and place them in your `_static/` folder. Make sure the filenames match exactly what you write in the template.

Styling text with CSS

You can style any text in a template using inline CSS or CSS classes. For example, to display a role name in bold red:

```
<span style="color: red; font-weight: bold;">Hunter 1</span>
```

Or in italic green:

```
<span style="color: green; font-style: italic;">Hunter 2</span>
```

For something you will reuse many times, define CSS classes at the top of your template:

```
<style>
    .role1 { color: red; font-weight: bold; }
    .role2 { color: green; font-style: italic; }
</style>
```

Then apply them using the `class` attribute:

```
{{ if player.role == C.PLAYER1_ROLE }}
    <span class="role1">{{ player.role }}</span>
{{ else }}
    <span class="role2">{{ player.role }}</span>
{{ endif }}
```

> **Inline styles vs. CSS classes**
>
> Inline styles (`style="color: red;"`) are fine for one-off use. But if you need the same style in many places — like colouring every mention of a role — CSS classes are much more efficient. Define the class once, then use `class="role1"` wherever you need it.

Step 11: test your changes

1. Reset the database:

```
otree resetdb
```

2. Start the server:

```
otree devserver
```

3. Open **two browser tabs** and go to `http://localhost:8000/demo`. Select the session that includes your modified app and click through both tabs simultaneously, simulating two players.
4. Verify the following:

- ☐ The introduction appears only in round 1
- ☐ The decision page appears in every round
- ☐ The game history table is hidden in round 1 and appears from round 2
- ☐ The history shows correct choices, payoffs, and cumulative totals
- ☐ The results page shows the current round's outcome plus the full history
- ☐ The game runs for 5 rounds total
- ☐ Each player sees their role on screen
- ☐ The timer counts down on the decision page
- ☐ If you let the timer expire, a default choice is made automatically
- ☐ Images display correctly on the results page

What you have learned

You have now made the key modifications that transform a one-shot game into a repeated game with history, roles, timers, and visual elements. These same patterns carry directly into your Stag Hunt assignment:

Modification	Prisoner's Dilemma → Stag Hunt
Change `NUM_ROUNDS`	`1` → up to `10` with a stopping rule
`is_displayed` for intro pages	Same pattern — show instructions in round 1 only
`creating_session` for grouping	Same — randomize in round 1, fix thereafter
`per_round_payoff` field	Same — track per-round earnings separately
Game history with `in_previous_rounds()`	Same loop pattern with different field names
History table in HTML	Same `{ for }` loop with different columns
Role constants and `player.role`	`PLAYER1_ROLE` → `HUNTER1_ROLE`, etc.
`timeout_seconds` + `before_next_page`	Same pattern for the decision timer
`{ static 'file' }` for images	Same — place files in `_static/`
CSS classes for styling	Same — define once, apply everywhere

What comes next

Your Stag Hunt assignment builds on everything covered in this session. The additional features you will need to implement on your own include:

- An **asymmetric payoff matrix** where the two roles face different payoffs for some outcomes
- A **payoff change mid-game** where a parameter changes after a certain round

- A **probabilistic stopping rule** that ends the game randomly after a minimum number of rounds
- A **random round for payment** selected at the end of the game
- A **custom wait page** that replicates the decision screen layout
- A **questionnaire** and **summary screen** as separate apps with data flowing between them
- **Session-level statistics** calculated across all groups

The next session's cheat sheet will give you a quick reference for the oTree tools and methods you will need for these features. Use it alongside the official oTree documentation and AI assistance to build your assignment.

Assignment 2 Cheat Sheet

This is a quick reference for the oTree features and Python patterns you will need for your Stag Hunt assignment. It is not a tutorial — it tells you **what exists** and **how to use it**, so you know what to reach for when building your app. Use this alongside the official oTree documentation and AI assistance.

Constants (`class C`)

Required constants

Every app must define these three:

```
class C(BaseConstants):
    NAME_IN_URL = 'my_app'       # Appears in participants' browser URL
    PLAYERS_PER_GROUP = 2        # None for individual tasks
    NUM_ROUNDS = 10              # How many times the page sequence repeats
```

Currency values

```
PAYOFF_A = cu(4)     # Creates a formatted currency value
PAYOFF_B = cu(0)     # oTree handles dollar signs, decimals, etc.
```

Role constants

```
HUNTER1_ROLE = 'Hunter_1'    # Assigned to id_in_group = 1
HUNTER2_ROLE = 'Hunter_2'    # Assigned to id_in_group = 2
```

Access in code: `player.role` returns the role string (e.g., `'Hunter_1'`).

Access in templates: `{ player.role }`.

Model fields

Field types

Type	Use for	Example
`models.BooleanField()`	True/False choices	Player's decision (A or B)
`models.IntegerField()`	Whole numbers	Age, number of courses
`models.FloatField()`	Decimal numbers	Probabilities, percentages
`models.StringField()`	Text	Name, major, open-ended response
`models.CurrencyField()`	Money amounts	Per-round payoff
`models.LongStringField()`	Long text	Essay responses, comments

Common field arguments

Argument	What it does	Example
`label`	Question text shown to participant	`label='What is your age?'`
`choices`	List of `[recorded, displayed]` pairs	`choices=[[True, 'A'], [False, 'B']]`
`widget`	How the field is rendered	`widget=widgets.RadioSelect`
`min` / `max`	Validation for numeric fields	`min=18, max=99`
`blank`	If `True`, field is optional	`blank=True`
`doc`	Description in data export	`doc="Player's decision"`
`default`	Default value	`default=0`

Where to define fields

Level	Class	When to use
Individual data	`class Player(BasePlayer)`	Decisions, payoffs, survey responses
Group data	`class Group(BaseGroup)`	Data about the pair/group as a whole
Round data	`class Subsession(BaseSubsession)`	Data that applies to the entire round

Built-in player attributes

These exist automatically on every player — you do not define them.

Attribute	What it returns
`player.id_in_group`	Position in the group (1, 2, …)
`player.role`	Role name (from constants, e.g., `'Hunter_1'`)
`player.round_number`	Current round (1, 2, …, up to `NUM_ROUNDS`)
`player.payoff`	Built-in earnings field (currency)

Accessing other players and rounds

Within the same round

Method	Returns
`player.get_others_in_group()`	List of other players in the same group
`player.group`	The Group object this player belongs to
`group.get_players()`	List of all players in the group
`group.get_player_by_role('Hunter_1')`	The specific player with that role
`subsession.get_players()`	All players in the round
`subsession.get_groups()`	All groups in the round

Across rounds

Method	Returns
`player.in_previous_rounds()`	List of this player's objects from earlier rounds
`player.in_all_rounds()`	List from all rounds including current
`player.in_round(3)`	This player's object in round 3 specifically
`group.in_all_rounds()`	List of this group's objects from all rounds
`group.in_round(3)`	This group's object in round 3 specifically

Display values

Method	Returns
`player.field_display('stag')`	The display label (e.g., `'A'`) instead of the raw value (`True`)

Grouping

In `creating_session`

```
def creating_session(subsession: Subsession):
    if subsession.round_number == 1:
        subsession.group_randomly()          # Random groups in round 1
    else:
        subsession.group_like_round(1)       # Same groups as round 1
```

Other grouping methods

Method	What it does
`subsession.group_randomly()`	Randomly assigns players to groups
`subsession.group_randomly(fixed_id_in_group=True)`	Random groups but keep same `id_in_group`
`subsession.group_like_round(n)`	Copy group assignments from round `n`

Page classes

Basic page

```
class MyPage(Page):
    form_model = 'player'                  # Which model the form saves to
    form_fields = ['field1', 'field2']      # Which fields appear on the page
```

Page methods

Method	When it runs	Common use
`is_displayed(player)`	Before the page is shown	Skip pages conditionally
`vars_for_template(player)`	Before rendering the template	Pass calculated data to HTML
`before_next_page(player, timeout_happened)`	After form submission	Set defaults, do calculations
`app_after_this_page(player, upcoming_apps)`	After the page	Skip to a different app entirely

```
class Decision(Page):
    form_model = 'player'
    form_fields = ['stag']
    timeout_seconds = 60

    @staticmethod
    def is_displayed(player):
        return player.round_number == 1

    @staticmethod
    def vars_for_template(player):
        return dict(my_variable=some_value)

    @staticmethod
    def before_next_page(player, timeout_happened):
        if timeout_happened:
            player.stag = True

    @staticmethod
    def app_after_this_page(player, upcoming_apps):
        if some_condition:
            return upcoming_apps[0]    # Skip to the next app
```

Wait pages

```
class MyWaitPage(WaitPage):
    after_all_players_arrive = my_function    # Runs once when all players arrive
```

Or with a method defined inline:

```
class MyWaitPage(WaitPage):
    @staticmethod
    def after_all_players_arrive(group):
        # Calculate payoffs, check stopping rule, etc.
        for p in group.get_players():
            set_payoff(p)
```

Custom wait page template

```
class MyWaitPage(WaitPage):
    template_name = __name__ + '/MyWaitPage.html'
    title_text = "Waiting for other participants..."

    @staticmethod
    def vars_for_template(player):
        return dict(my_data=some_value)
```

Wait pages can also have `vars_for_template` to pass data to the custom template.

Timers

```
class MyPage(Page):
    timeout_seconds = 120          # Fixed timer (2 minutes)
    timeout_seconds = C.MY_TIMER   # Timer from a constant
```

The timer appears automatically on the page and counts down. When it reaches zero, the page submits automatically. On pages with form fields, always pair with `before_next_page` to handle missing data.

Cross-app data: `PARTICIPANT_FIELDS` and `SESSION_FIELDS`

Why you need them

Player and group data only exist within their app. To pass data from one app to the next, you store it on the `participant` (for individual data) or `session` (for shared data).

Declaring fields in `settings.py`

```
PARTICIPANT_FIELDS = ['finished_rounds', 'round_to_pay', 'opponent_payoff']
SESSION_FIELDS = ['total_rounds_played', 'yes_stag']
```

You must list every custom participant/session variable here. Without this declaration, oTree will raise an error.

Setting values

```
# Participant-level (specific to one person, persists across apps)
player.participant.round_to_pay = 5
player.participant.finished_rounds = True

# Session-level (shared across everyone, persists across apps)
player.session.total_rounds_played = 0
player.session.yes_stag = 0
```

Reading values in another app

```
# In a later app's vars_for_template or page method
payment_round = player.participant.round_to_pay
total_stag = player.session.yes_stag
```

In templates:

```
{{ participant.round_to_pay }}
{{ session.total_rounds_played }}
```

Initializing in `creating_session`

Session-level variables should be initialized once (typically in round 1 of the first app):

```
def creating_session(subsession):
    if subsession.round_number == 1:
        subsession.session.my_counter = 0
```

Skipping rounds and apps

Skipping to the next app with `app_after_this_page`

```
@staticmethod
def app_after_this_page(player, upcoming_apps):
    if player.participant.finished_rounds:
        return upcoming_apps[0]    # Jump to the next app in app_sequence
```

`upcoming_apps` is a list of app names remaining in `app_sequence`. Returning `upcoming_apps[0]` sends the player to the very next app, skipping all remaining rounds.

Return `None` (or do not return anything) to continue normally.

Template syntax

Displaying data

```
{{ player.payoff }}              <!-- Built-in field -->
{{ player.role }}                <!-- Built-in attribute -->
{{ C.NUM_ROUNDS }}               <!-- Constant -->
{{ my_variable }}                <!-- From vars_for_template -->
{{ player.round_number }}        <!-- Current round number -->
```

Conditionals

```
{{ if player.round_number == 1 }}
    <p>Welcome!</p>
{{ endif }}

{{ if same_choice }}
    <p>You agreed.</p>
{{ else }}
    <p>You disagreed.</p>
{{ endif }}

{{ if player.role == 'Hunter_1' }}
```

```
    <b style="color: red;">Hunter 1</b>
{{ else }}
    <i style="color: green;">Hunter 2</i>
{{ endif }}
```

Loops

```
{{ for row in game_history }}
<tr>
    <td>{{ row.round }}</td>
    <td>{{ row.my_choice }}</td>
</tr>
{{ endfor }}
```

Forms

```
{{ formfields }}                            <!-- All fields at once -->
{{ formfield 'stag' }}                      <!-- One specific field -->
{{ formfield 'stag' label="Your choice" }} <!-- With custom label -->
```

Buttons and navigation

```
{{ next_button }}              <!-- Standard "Next" button -->

<!-- Custom button that submits a form field -->
<button name="stag" value="True" class="btn btn-primary">
    Choose A
</button>
```

Including other files

```
{{ include_sibling 'instructions.html' }}
```

Static files (images, CSS, JS)

```
<img src="{{ static 'happy.png' }}" alt="Happy" width="80">
```

Place files in the `_static/` folder at the project level.

The `settings.py` session config

```
SESSION_CONFIGS = [
    dict(
        name='my_experiment',
        display_name='My Stag Hunt Experiment',
        app_sequence=['stag_hunt', 'questionnaire', 'summary'],
        num_demo_participants=2,
    ),
]
```

Key	What it does
`name`	Internal identifier (no spaces)
`display_name`	Shown in the admin interface
`app_sequence`	Ordered list of app names participants go through
`num_demo_participants`	How many demo players to create (use a multiple of `PLAYERS_PER_GROUP`)

Python patterns quick reference

If / elif / else

```
if player.round_number <= 3:
    X = cu(2)
elif player.round_number <= 6:
    X = cu(3)
else:
    X = cu(4)
```

For loops

```
for p in group.get_players():
    print(p.role, p.payoff)

for p in player.in_previous_rounds():
    total += p.per_round_payoff
```

Dictionaries

```
# Create
payoff_matrix = {
    (True, True): cu(4),
    (True, False): cu(0),
    (False, True): cu(1),
    (False, False): cu(1),
}

# Look up
result = payoff_matrix[(player.stag, other.stag)]
```

Lists

```
history = []                          # Create empty list
history.append({'round': 1})          # Add item
first = history[0]                    # Access by index
length = len(history)                 # Get length
```

List comprehensions

```
# Create a list from a loop in one line
payoffs = [p.per_round_payoff for p in player.in_all_rounds()]
total = sum(payoffs)
```

Random numbers

```
import random
random.random()                        # Float between 0 and 1
random.choice([1, 2, 3, 4, 5])        # Pick one random item
random.choice(range(1, 11))           # Pick random int from 1 to 10
```

String formatting

```
message = f"Round {player.round_number} of {C.NUM_ROUNDS}"
```

The += operator

```
total = 0
total += 5     # total is now 5
total += 3     # total is now 8
```

Debugging

`print()` in the terminal

Insert `print()` anywhere in your Python code. Output appears in the terminal where `otree devserver` is running.

```
print('Player role:', player.role)
print('Stag choice:', player.stag)
print('Payoff:', player.per_round_payoff)
```

Remove or comment out `print()` statements before deploying.

Common errors and fixes

Error	Likely cause	Fix
"Table has no column named X"	Added a field but did not reset database	Run `otree resetdb`
"NoneType has no attribute"	A field is `None` (not set)	Check timeout handling, check field defaults
Page is blank / crashes	Template name does not match page class	File must be `PageClassName.html` exactly
Stuck on wait page	Only one browser tab open	Open a second tab to simulate the other player
"X is not defined" in template	Variable not in `vars_for_template`	Add it to the `return dict(...)`
Session/participant field error	Field not declared in `settings.py`	Add to `SESSION_FIELDS` or `PARTICIPANT_FIELDS`

Useful links

- Official oTree documentation
- oTree sample games (source code)
- oTree snippets (common recipes)
- oTree Hub (featured projects)
- New oTree forum

Session 3

Extra Features, HTML Templates, and Cross-App Data

In Session 2, you learned how the core game logic works inside `__init__.py` — constants, groups, payoff calculations, page flow, timers, and the stopping rule. But `__init__.py` is only half the picture. The other half is the **HTML templates** — the files that control what participants actually see on their screens.

This session covers three areas. First, how to build and style HTML templates for your oTree pages. Second, how to implement the extra features required by the assignment (role-specific styling, custom wait pages, images on results pages, and session-level statistics). Third, how data flows across the three apps in the session — from the Stag Hunt game to the questionnaire to the final summary screen.

HTML templates in oTree

Every page class in your `__init__.py` needs a corresponding HTML file. oTree looks for a file with the exact same name as the page class, inside the app's folder. For example:

Page class in `__init__.py`	Template file
`Introduction`	`assignment/Introduction.html`
`Decision`	`assignment/Decision.html`
`ResultSummary`	`assignment/ResultSummary.html`

File names must match exactly

The template file name must match the page class name exactly — including capitalisation. `Decision.html` works. `decision.html` does not. If oTree cannot find the template, the page will crash.

Basic template structure

Every oTree HTML template follows this basic structure:

```
{{ block title }}
  Your Page Title
{{ endblock }}

{{ block content }}
  <p>Everything the participant sees goes here.</p>

  {{ next_button }}
{{ endblock }}
```

The `{ block title }` section sets the page title (shown at the top of the page and in the browser tab). The `{ block content }` section contains the actual page content. `{ next_button }` renders the standard "Next" button that advances the participant to the next page.

Displaying variables

Any variable passed through `vars_for_template` or automatically available (like `player`, `group`, `C`) can be displayed using double curly braces:

```
<p>Your role is: {{ player.role }}</p>
<p>The current round is: {{ player.round_number }} of {{ C.NUM_ROUNDS }}</p>
<p>The value of X is: {{ X }}</p>
```

Displaying form fields

On pages with `form_fields`, you display each field using the `{ formfield }` tag or `{ formfields }` to display all fields at once:

```
{{ formfields }}
```

Or individually:

```
{{ formfield 'stag' }}
```

Using `if` statements in templates

oTree templates support conditional logic using `{ if }`:

```
{{ if player.round_number == 1 }}
  <p>Welcome to the first round!</p>
{{ endif }}

{{ if same_choice }}
  <p>You and your partner made the same choice.</p>
{{ else }}
  <p>You and your partner made different choices.</p>
{{ endif }}
```

Using `for` loops in templates

To display the game history table, you loop through the `game_history` list:

```
<table class="table">
  <thead>
    <tr>
      <th>Round</th>
      <th>Your Choice</th>
      <th>Your Payoff</th>
      <th>Partner's Choice</th>
      <th>Partner's Payoff</th>
      <th>Cumulative Payoff</th>
    </tr>
  </thead>
  <tbody>
    {{ for row in game_history }}
    <tr>
      <td>{{ row.0 }}</td>
      <td>{{ row.1 }}</td>
      <td>{{ row.2 }}</td>
      <td>{{ row.3 }}</td>
      <td>{{ row.4 }}</td>
      <td>{{ row.5 }}</td>
    </tr>
    {{ endfor }}
  </tbody>
</table>
```

Each `row` is one of the lists appended to `game_history` in `vars_for_template`. The `.0`, `.1`, `.2`, etc. notation accesses the first, second, third elements of that list (Python uses zero-based indexing).

> **Bootstrap is included by default**
>
> oTree includes Bootstrap, a CSS framework that provides pre-styled components. The `class="table"` attribute above uses Bootstrap's table styling to make the table look clean without any custom CSS. You can use any Bootstrap classes in your templates.

Extra feature: role-specific styling

The assignment requires "all references to Hunter 1 made with bolded red font, and all references to Hunter 2 made with italicized green font." You can achieve this with HTML inline styles or CSS classes.

Approach 1: inline styles

```
{{ if player.role == 'Hunter_1' }}
  <p>You are <b style="color: red;">Hunter 1</b>.</p>
{{ else }}
  <p>You are <i style="color: green;">Hunter 2</i>.</p>
{{ endif }}
```

Approach 2: CSS classes (cleaner)

At the top of your template, define CSS classes:

```
<style>
  .hunter1 { color: red; font-weight: bold; }
  .hunter2 { color: green; font-style: italic; }
</style>
```

Then use them throughout:

```
<p>You are <span class="{{ if player.role == 'Hunter_1' }}hunter1{{ else }}hunter2{{ endif }}
  {{ player.role }}
</span>.</p>
```

The CSS approach is better because you define the styling once and reuse it everywhere. If you need to change the colour, you change it in one place.

Extra feature: custom wait page

The assignment asks for a waiting screen that is "an exact replica of the decision screen but with no ability to input a decision." This is why the `ResultsWaitPage` has:

```
template_name = __name__ + '/ResultsWaitPage.html'
```

The custom template file `ResultsWaitPage.html` displays the payoff matrix and game history table (using the same `vars_for_template` data) but without any form fields or submit button. The participant can review the game state while waiting.

> Building the custom wait page template
>
> Create a file called `ResultsWaitPage.html` in your app folder. It should contain the same payoff matrix and game history table as `Decision.html`, but without `{ formfields }` or `{ next_button }`. You should include the standard oTree wait page JavaScript to handle the auto-advance when the other player arrives. oTree's documentation on custom wait pages provides the necessary template code.

Extra feature: images on results pages

The assignment asks for "an image of a happy face if both chose Stag and a sad face otherwise." This combines a template `if` statement with an HTML image tag.

First, place your image files (e.g., `happy.png` and `sad.png`) in the `_static/` folder of your project. Then in the results template:

```
{{ if same_choice }}
  <img src="{{ static 'happy.png' }}" alt="Happy face" width="100">
{{ else }}
  <img src="{{ static 'sad.png' }}" alt="Sad face" width="100">
{{ endif }}
```

The `{ static 'filename' }` tag generates the correct URL for files in the `_static/` folder. This is the standard way to include images, CSS files, or JavaScript files in oTree templates.

> **The `_static` folder**
>
> All shared static files — images, custom CSS, JavaScript — go in the `_static/` folder at the project level (not inside any individual app). oTree serves these files automatically.

Extra feature: Stag-Stag percentage on the summary screen

The session summary app calculates the percentage of rounds where both players chose Stag across all groups:

```
stag_stag_percentage = (player.session.yes_stag / player.session.total_rounds_played) * 100
```

This uses the session-level variables that were accumulated during the game:

- `player.session.yes_stag` — the total number of Stag-Stag outcomes across all groups
- `player.session.total_rounds_played` — the total number of rounds played across all groups

Dividing the first by the second gives the proportion, and multiplying by 100 converts it to a percentage.

Cross-app data flow

The three-app session (`assignment` → `questionnaire_assignment` → `session_summary_assignment`) relies on data being passed between apps. Here is how that works.

What persists across apps automatically

The `participant` object persists across all apps. Any variable stored on `participant` (and declared in `PARTICIPANT_FIELDS`) is available in every app. The `session` object also persists, and any variable declared in `SESSION_FIELDS` is available everywhere.

Data flow in this experiment

Variable	Set in	Used in	Level
`participant.finished_assignment`	`assignment` (wait page)	`assignment` (`app_after_this_page`)	Participant
`participant.round_to_pay`	`payment` (wait page)	`session_summary_assignment`	Participant
`participant.opponent_payoff`	`payment` (wait page)	`session_summary_assignment`	Participant
`session.total_rounds_played`	`payment` (wait page)	`session_summary_assignment`	Session
`session.yes_stag`	`assignment` (wait page)	`session_summary_assignment`	Session

The questionnaire app

The `questionnaire_assignment` app is a simple demographics survey — identical in structure to the one from Session 1. It collects age, gender, race, ethnicity, field of study, and language data. Because it has `PLAYERS_PER_GROUP = None`, it does not need groups and participants complete it independently.

The session summary app

The `session_summary_assignment` app displays the final results. Its `vars_for_template` pulls in participant-level and session-level data:

```
return dict(
    stag_stag_percentage = stag_stag_percentage,
    payment_round = player.participant.round_to_pay,
    session_payoff = player.participant.payoff_plus_participation_fee(),
    today_date = player.today_date,
)
```

- **`player.participant.round_to_pay`** — The randomly selected round, set in the Stag Hunt app.
- **`player.participant.payoff_plus_participation_fee()`** — A built-in oTree method that adds the participation fee (from `SESSION_CONFIG_DEFAULTS`) to the player's game earnings. This is the total amount the participant receives.
- **`player.today_date`** — The current date, formatted using Python's `datetime` library.

The `datetime` library

The session summary app uses Python's built-in `datetime` library to display the current date:

```
from datetime import datetime
current_datetime = datetime.now()
player.today_date = current_datetime.strftime("%m-%d-%Y")
```

`datetime.now()` gets the current date and time. `.strftime("%m-%d-%Y")` formats it as month-day-year (e.g., "03-31-2026"). The `import` statement at the top of the file makes the `datetime` class available.

Building the payoff matrix in HTML

One of the most important elements of the decision screen is the payoff matrix — a table showing all possible outcomes. Here is an example of how to build it in HTML:

```
<table class="table table-bordered text-center">
  <thead>
    <tr>
      <th></th>
      <th></th>
      <th colspan="2">Partner's Choice</th>
    </tr>
    <tr>
      <th></th>
      <th></th>
      <th>A</th>
      <th>B</th>
    </tr>
  </thead>
  <tbody>
    <tr>
      <th rowspan="2">Your Choice</th>
      <th>A</th>
      <td>{{ C.PAYOFF_B }}, {{ C.PAYOFF_B }}</td>
      <td>{{ C.PAYOFF_D }}, {{ X }}</td>
    </tr>
    <tr>
```

```
      <th>B</th>
      <td>{{ C.PAYOFF_A }}, {{ X }}</td>
      <td>{{ C.PAYOFF_A }}, {{ X }}</td>
    </tr>
  </tbody>
</table>
```

> Use constants from `C` in your templates
>
> Notice how the table uses `{ C.PAYOFF_B }`, `{ C.PAYOFF_D }`, etc. instead of hard-coded numbers. This way, if you ever change the payoff values in your constants, the template updates automatically.

Summary

Concept	What it does
HTML templates	Control what participants see on each page
`{ variable }`	Displays a variable's value in the template
`{ formfields }`	Renders form fields on pages with `form_fields`
`{ if } ... {{ endif }}`	Conditional display in templates
`{ for } ... {{ endfor }}`	Loops through lists to display repeated content
`{ static 'file' }`	Links to files in the `_static/` folder
`{ next_button }`	Renders the standard "Next" button
CSS classes	Style elements consistently across pages
`_static/` folder	Stores shared images, CSS, and JavaScript
`PARTICIPANT_FIELDS`	Declares participant-level variables for cross-app data
`SESSION_FIELDS`	Declares session-level variables for cross-app data
`participant.payoff_plus_participation_fee()`	Calculates total payment including show-up fee
`datetime`	Python library for formatting dates

Session 4

Using Claude for Coding and Deploying Your Experiment

You have now learned how to build a complete oTree experiment — from the data model and payoff logic to HTML templates and cross-app data flow. This final session covers two remaining topics. First, how to use AI tools like Claude effectively to help you write, debug, and improve your oTree code. Second, how to package your project and deploy it on a server so you can run a live session with real participants.

Using Claude to help with oTree coding

The assignment states: "You should also make heavy use of an AI LLM (ChatGPT, Claude, etc). Learning how to leverage AI assistance will be an essential part of creating your best code." This section teaches you how to do that well.

What AI can do for you

AI assistants are excellent at several tasks that come up constantly in oTree development:

- **Generating boilerplate code** — "Write an oTree page class with a timer and a form field called `contribution`."
- **Explaining code** — "What does `player.in_previous_rounds()` return?"
- **Debugging errors** — Paste your error traceback and ask what went wrong.
- **Writing HTML templates** — "Build an HTML table that displays a 2×2 payoff matrix."
- **Translating logic into code** — "I want to randomly select one round for payment at the end of the game."
- **Refactoring** — "Can you rewrite this chain of if/else statements as a dictionary lookup?"

How to write effective prompts

The quality of the code you get back depends on the quality of your prompt. Here are principles that work well for oTree coding.

Be specific about the oTree version. oTree has changed significantly over the years. The newer "no-self" format (used in this course) is different from older versions. Always mention that you are using the current oTree syntax. For example: "I am using the latest version of oTree with the no-self format. Write a function that…"

Provide context. Do not just ask "how do I calculate payoffs?" Instead, describe your game: "I have a two-player Stag Hunt with `PLAYERS_PER_GROUP = 2`. Each player chooses A or B. I need a function that looks up the payoff from a 2×2 matrix based on both players' choices."

Share your existing code. If you have a partial implementation or a function that is not working, paste the relevant section and explain what you expected versus what happened. The more context the AI has, the better its suggestions will be.

Ask for explanations, not just code. Instead of "write this for me," try "write this for me and explain what each line does." This helps you learn and makes it easier to modify the code later.

Iterate. If the first response is not quite right, explain what needs to change. "This is close, but the payoff matrix should be different for Hunter 1 and Hunter 2. Hunter 2's B payoff is X, which changes after round 3."

Example prompt for an oTree task

Here is a prompt that would produce good results:
"I am building an oTree experiment using the latest no-self format. I have a two-player game with `PLAYERS_PER_GROUP = 2` and `NUM_ROUNDS = 10`. I want to implement a random stopping rule: after round 5, each round should have a 55% chance of being the last round, but the game should never go past round 10. When the game ends, I want to randomly select one round for payment. Can you write the `after_all_players_arrive` method for my WaitPage that implements this? Please explain what each part does."
This prompt specifies the oTree version, the game structure, the exact rule to implement, where the code should go, and asks for explanations.

What AI is not good at

AI is a powerful tool but it has real limitations:

- **It can produce code that looks correct but has subtle bugs.** Always test your code by running `otree devserver` and clicking through the experiment yourself.
- **It may use outdated oTree syntax.** If you see `self.player` or `self.session` in generated code, that is the old format. The correct syntax is `player` and `player.session`.
- **It cannot test your experiment.** It does not know if your payoffs display correctly, if your timer works, or if your wait page releases both players at the right time. Only running the code can tell you that.

- **It does not know your experimental design.** You need to check that the generated code implements the game you intend, not just a game that runs without errors.

> ⚠ Always test AI-generated code
>
> Never submit code you have not tested yourself. Run `otree devserver`, open two browser tabs (to simulate two players), and play through the entire experiment. Check that payoffs are correct, the game history table displays properly, the stopping rule works, and the summary screen shows the right information.

Debugging with `print()`

When your code does not work as expected, `print()` is your most useful debugging tool. You can insert `print()` statements anywhere in your Python code to see what values variables hold at that point:

```
def set_payoff(player: Player):
    other = other_player(player)
    print('Player role:', player.role)
    print('Player chose:', player.stag)
    print('Other chose:', other.stag)
    # ... rest of the function
```

The output appears in the terminal where `otree devserver` is running. This is enormously helpful for tracking down bugs — if a payoff is wrong, printing the inputs to the payoff calculation will usually reveal the problem.

> 💡 Remove `print()` before deploying
>
> `print()` statements are for development only. They clutter the server logs during real sessions. Remove or comment them out before deploying your experiment. You can comment out a line by adding `#` at the beginning:
>
> ```
> # print('Player role:', player.role)  # commented out for deployment
> ```

Common Python patterns for oTree

Here is a quick reference for the Python patterns you will use most often in oTree.

If/elif/else

```
if player.round_number <= 3:
    X = cu(2)
elif player.round_number <= 6:
    X = cu(3)
else:
    X = cu(4)
```

`elif` (short for "else if") lets you chain multiple conditions. Python checks them in order and runs the first one that is `True`.

For loops

```
# Loop through all players in a group
for p in group.get_players():
    print(p.role, p.stag)

# Loop through all previous rounds
for p in player.in_previous_rounds():
    print(p.round_number, p.per_round_payoff)

# Loop with index using enumerate
for i, p in enumerate(group.get_players()):
    print(f"Player {i}: {p.role}")
```

Lists

```
# Create an empty list and add items
history = []
history.append('round 1 data')
history.append('round 2 data')

# Access items by index (starts at 0)
first_item = history[0]
```

```
# List comprehension - create a list from a loop
payoffs = [p.per_round_payoff for p in player.in_all_rounds()]
total = sum(payoffs)
```

Dictionaries

```
# Create a dictionary
payoff_matrix = {
    (True, True): cu(4),
    (True, False): cu(0),
    (False, True): cu(1),
    (False, False): cu(1),
}

# Look up a value
result = payoff_matrix[(True, False)]  # returns cu(0)
```

String formatting

```
# f-strings (the modern way)
name = "Hunter 1"
payoff = cu(4)
message = f"{name} earned {payoff} this round."
```

Deploying your experiment

Once your code is tested and working locally, you need to deploy it to a server so that real participants can access it over the internet.

Step 1: Zip your project

In the terminal, navigate to your project folder and run:

```
otree zip
```

This creates a `.otreezip` file containing your entire project. This is the file you will upload to oTree Hub.

> ⚠ Check your `settings.py` before zipping
>
> Before zipping, make sure:
>
> - `DEBUG = 0` — Do not deploy with debug mode on.
> - Your `SESSION_CONFIGS` list contains the session you want to run.
> - `SESSION_FIELDS` and `PARTICIPANT_FIELDS` are correctly declared.
> - `participation_fee` and `real_world_currency_per_point` are set to the correct values.

Step 2: Upload to oTree Hub

1. Go to oTree Hub and log in (or create an account).
2. Create a new project on oTree Hub.
3. Upload your `.otreezip` file.
4. oTree Hub will build and deploy your project.

Step 3: Connect to Heroku

oTree Hub can deploy your project to Heroku, a cloud platform that hosts your experiment and makes it accessible via a URL. Follow the oTree Hub instructions to link your Heroku account and deploy.

Once deployed, you will receive a URL like `https://your-project-name.herokuapp.com`. This is the address participants will use to access the experiment.

Step 4: Set up rooms for a live session

For running sessions in a lab, oTree uses **rooms**. A room assigns a fixed URL to each participant station, which prevents the problems that can occur with open session-wide links (like the same participant joining twice).

To create a room:

1. Create a text file in your project's `_rooms/` folder — for example, `_rooms/econ227b.txt`.
2. List one participant label per line (e.g., `P1`, `P2`, `P3`, …, `P20`).
3. In `settings.py`, add a `ROOMS` setting that references this file.

When you start a session using a room, each participant label gets its own unique link. In the lab, each computer opens its assigned link.

> **Test your deployment**
>
> After deploying to Heroku, always run a complete test session through the deployed URL — not just locally. Network latency, server configuration, and browser differences can all cause issues that do not appear on `localhost`.

Step 5: Run the session

On the admin interface of your deployed oTree server:

1. Click **Sessions** → **Create new session**.
2. Select the session config you want to run (e.g., `assignment`).
3. Set the number of participants.
4. Choose a room if you set one up.
5. Click **Create**.

The admin interface will show you the participant links and allow you to monitor the session in real time — you can see which page each participant is on, who is waiting, and when everyone has finished.

Final checklist before submission

Before submitting your assignment, go through this checklist:

- ☐ All three apps run without errors (`otree devserver`)
- ☐ Payoffs are calculated correctly for both Hunter 1 and Hunter 2
- ☐ Payoffs change correctly after round 3 (X goes from 2 to 3)
- ☐ Groups of 2 are formed and remain fixed across rounds
- ☐ The stopping rule works (game continues after round 5 with probability 0.45, never past round 10)
- ☐ Game history table displays correctly on the decision screen
- ☐ Instructions are clear, use cold language, and display at 24pt font
- ☐ Hunter 1 references are in **bold red**, Hunter 2 references are in *italic green*
- ☐ Wait page replicates the decision screen without input ability
- ☐ Happy/sad face images appear on the results page
- ☐ Stag-Stag percentage appears on the final summary screen
- ☐ Timer works on the decision page, defaulting to Stag on timeout
- ☐ Random round for payment is selected and displayed

- ☐ Questionnaire collects at least 5 demographic questions
- ☐ `DEBUG = 0` before deployment
- ☐ Project zips successfully with `otree zip`
- ☐ All files uploaded to the Google Drive folder

Summary

Topic	Key takeaway
AI for coding	Be specific, provide context, share code, ask for explanations, always test
`print()` debugging	Insert print statements to inspect variable values in the terminal
If/elif/else	Chain conditions; Python checks them in order
For loops	Iterate through players, rounds, or any list
Lists and list comprehensions	Build data structures for game history and aggregation
Dictionaries	Use as payoff matrices with tuple keys
`otree zip`	Packages your project for upload
oTree Hub + Heroku	Deploys your project to a public URL
Rooms	Assign fixed participant links for lab sessions
Pre-deployment checks	`DEBUG = 0`, correct settings, full test run

Useful Resources

This is a curated list of resources to support your learning throughout this course. You do not need to master all of these — bookmark what is useful and return to it as needed.

oTree

The official oTree documentation is the most reliable resource for all questions about oTree. It is well-written, actively maintained, and covers fundamental concepts of what you will need.

Always check oTree's demo apps and common task recipes *before* you start writing code from scratch — sample code has likely already been written for what you are trying to do. The **Featured Projects** section on oTree Hub is a great place to start:

- **Game Theory Sample Games** — Classic games from economics and game theory (Prisoner's Dilemma, etc.). Useful for understanding how standard experiments are structured in oTree. Includes a live demo and source code.
- **More Demos** — A wide range of demo apps from different disciplines. A good starting point for finding code that is close to what you want to build. Includes a live demo and source code.
- **Snippets** — A collection of small recipes for common tasks in oTree. If you need to do something specific, check here first. Includes a live demo and source code(https://github.com/oTree-org/otree-snippets).

Both the old and the new oTree forums are useful sources of solutions to common problems. Note that the old forum contains solutions written in oTree's older `self` format, so some details may need to be adapted to the newer `no-self` format used in this course. See here for more details on how the new format differs from the old one.

Finally, the **Public projects** section on oTree Hub hosts code from other oTree users that is shared publicly.

Here is the complete list of oTree resources:

- Official oTree documentation
- Featured Projects on oTree Hub
- oTree's "Where do I put this code?" breakdown

- New oTree forum
- Old oTree forum
- Public projects

> Searching for oTree help
>
> When you run into a problem and are not sure where to start, try Googling your task or error message together with the word **"otree"** — for example, *"otree timer page"* or *"otree wait for all players"*. This will surface relevant results from the documentation, forums, and other oTree users.
> If you find a result on **Stack Overflow**, read the full discussion before implementing any solution — do not just copy the first answer you see. The comments often reveal important nuance, and your use-case may require a slightly different approach.

Python

Python is the backbone of your oTree app — it handles the logic, stores data, and communicates between the server and the participant's browser. A quick tip: use `print()` to see variable values and outputs in the terminal while you are debugging your code.

Web-based editors (for testing small bits of code)

The following editors are useful when you want to quickly test a small piece of Python code without running it through oTree. This list is adapted from resources originally compiled by Faith Bague-Sampson.

- Python Fiddle
- myCompiler: Python
- PYnative Online Python Editor

References